

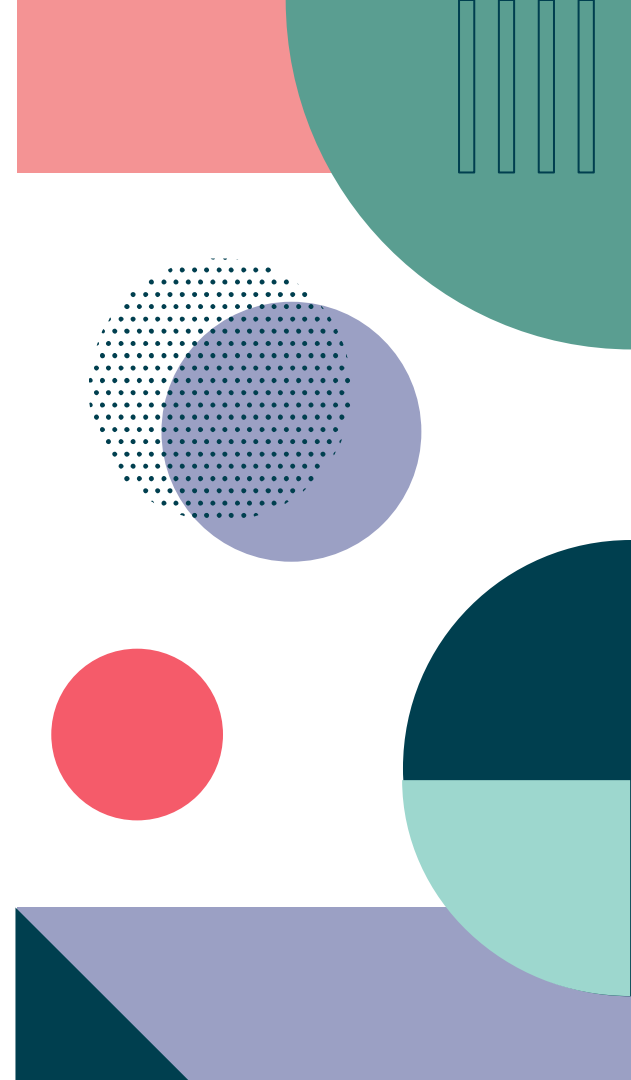


Agile Fundamentals

Let's start our journey with Agile

Heba ElAtfy

Agile
Transformation
Coach



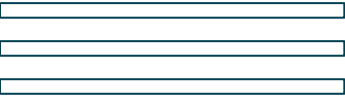
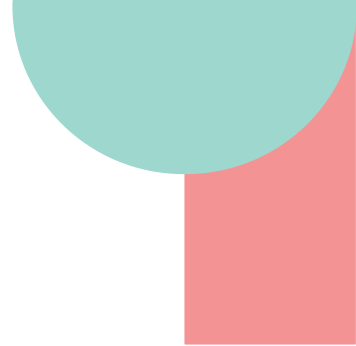


Table of Contents



1 Traditional Software Delivery

2 Agile History

3 What is Agile?

4 Lean

5 Kanban

6 SCRUM

7 Extreme Programming (XP)

8 Teams

9 Definitions

10 User Stories

11 Estimation

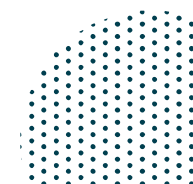
12 Retrospectives

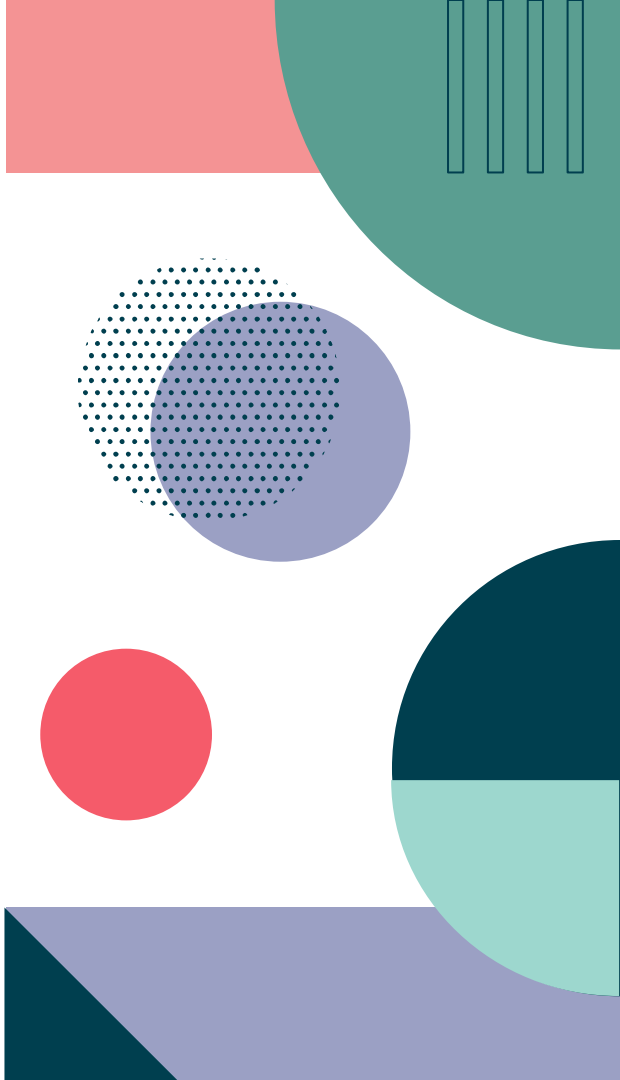
13 Technical Excellence

14 DevOps

15 Design Thinking

16 Q & A



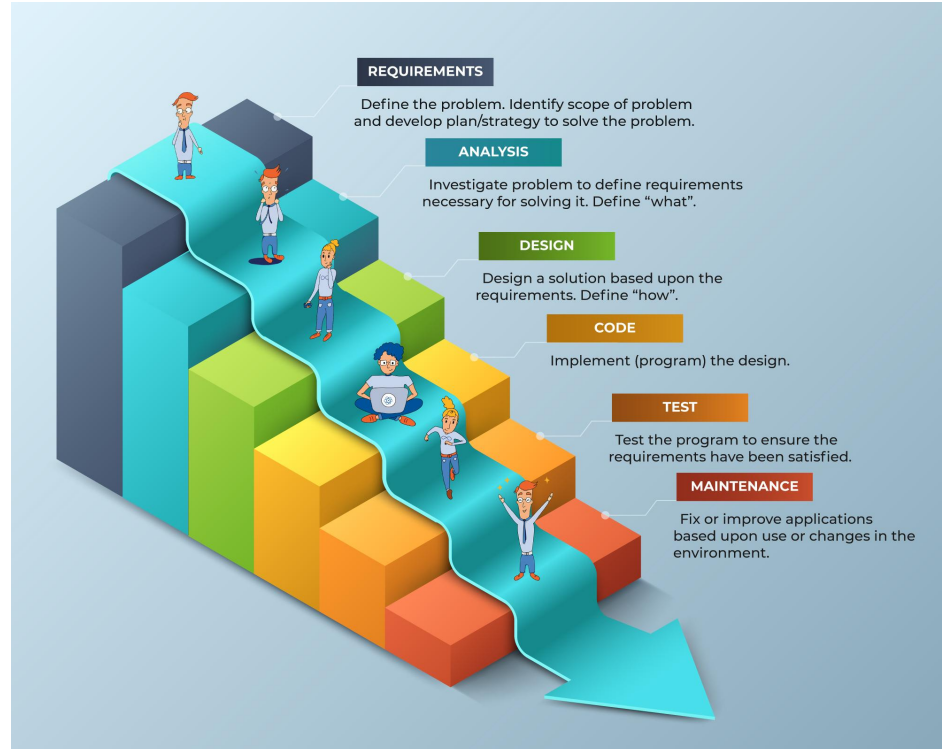


1

Traditional Software Development

Waterfall Software Delivery

- Sequential Development Process
- Full Project Planning upfront
- Huge time to deliver
- Customer not involved
- Hard to accommodate change request
- Deadline Creep (Phase delayed = Whole Project delayed)



Waterfall Advantages & Disadvantages

Advantages

1. Before the next phase of development, each phase must be completed
2. Suited for smaller projects where requirements are well defined
3. They should perform quality assurance test (Verification and Validation) before completing each stage
4. Elaborate documentation is done at every phase of the software development cycle
5. Project is completely dependent on project team with minimum client intervention
6. Any changes in software is made during the process of the development

Disadvantages

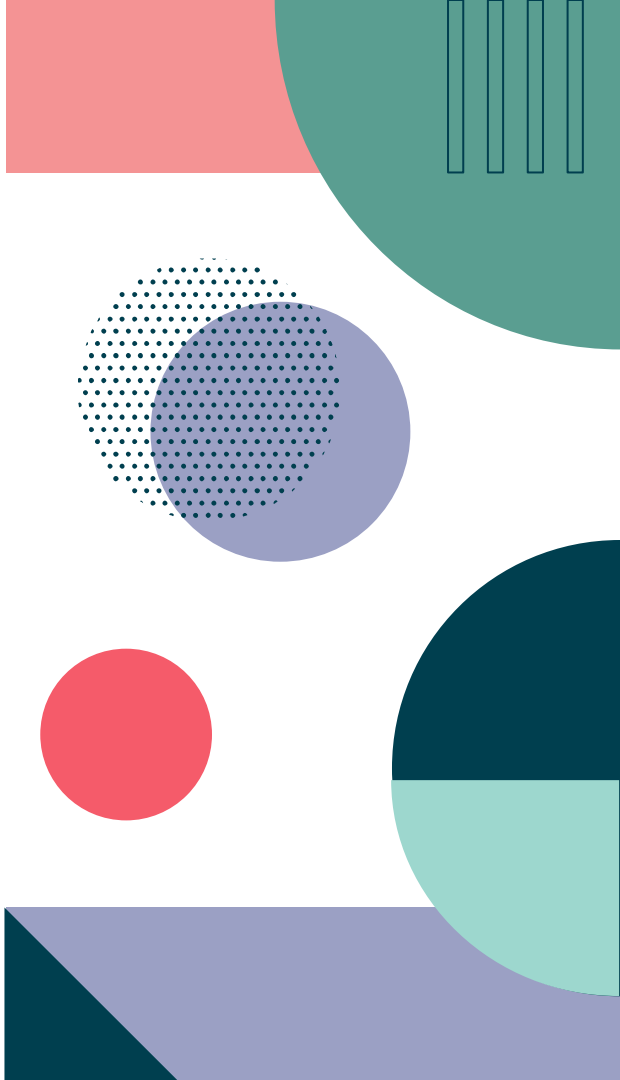
1. Error can be fixed only during the phase
2. It is not desirable for complex project where requirement changes frequently
3. Testing period comes quite late in the developmental process
4. Documentation occupies a lot of time of developers and testers
5. Clients valuable feedback cannot be included with ongoing development phase
6. Small changes or errors that arise in the completed software may cause a lot of problems

Waterfall Origin

Waterfall was supposedly “invented” by the paper “Managing the Development of Large Software Systems” by Dr. Winston W. Royce in 1970.

Actually, it is a paper identifying this pattern (without naming it Waterfall) and describing its deficiencies and proposing enhancements

*‘I believe in this concept,
but the implementation
described above is risky
and invites failure.’
Dr. Royce said*



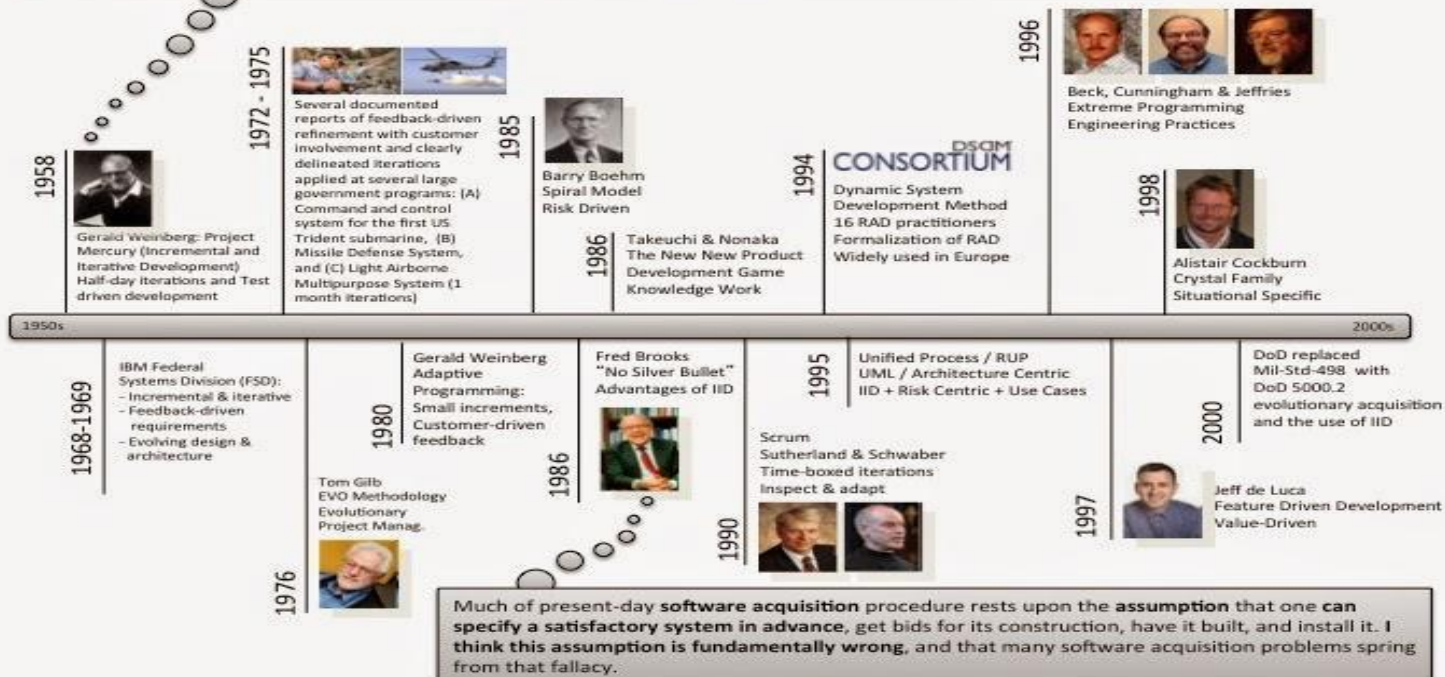
2

Agile History

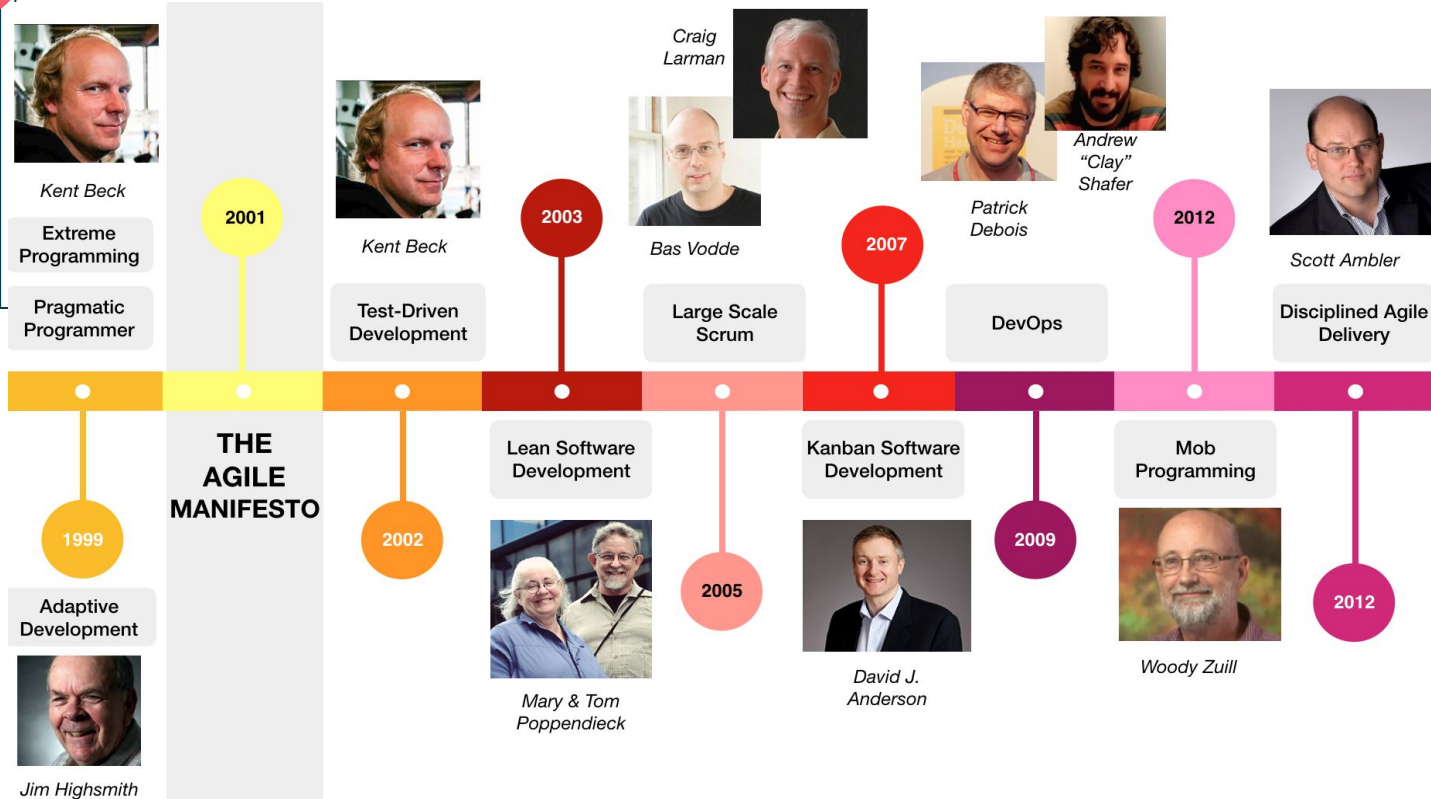
History, Principle &
Manifesto

Agile History

We were doing incremental development as early as 1957, in Los Angeles, under the direction of Bernie Dimsdale [at IBM's Service Bureau Corporation]. He was a colleague of **John von Neumann**, so perhaps he learned it there, or **assumed it as totally natural**. I do remember Herb Jacobs (primarily, though we all participated) developing a large simulation for Motorola, where the technique used was, as far as I can tell, **indistinguishable from XP**.



Recent Agile History



Agile Principles & Manifesto

A group of 17 people thought, “We’re all doing these different approaches to developing software. We ought to get together and see where there are commonalities in what we’re thinking about.” The result was a meeting at a ski resort in Snowbird, Utah in 2001.



Agile Manifesto

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, **we value the items on the left more.**

Agile Principles

01 Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

02 Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.

03 Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

04 Business people and developers must work together daily throughout the project.

05 Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

06 Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

07 Working software is the primary measure of progress.

08 The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

09 Continuous attention to technical excellence and good design enhances agility.

10 Simplicity – the art of maximizing the amount of work not done – is essential.

11 The best architectures, requirements, and designs emerge from self-organizing teams.

12 At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

*Take 10 min
to read the
12 principles
& simplify
them each
to 3 main
keywords*

Agile Principles (cont.)

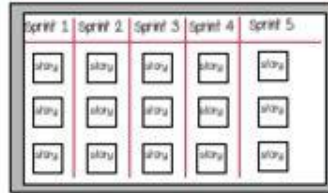
1 Satisfy the **customer**



2 Welcome **change**



3 Deliver **frequently**



4 Work **together**



5 Trust and **support**



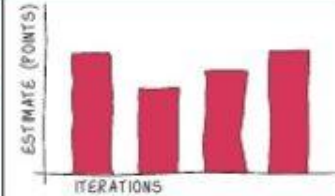
6 Face-to-face **conversation**



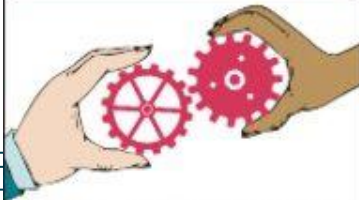
7 Working **software**



8 Sustainable **development**



9 Continuous **attention**



10 Maintain **simplicity**

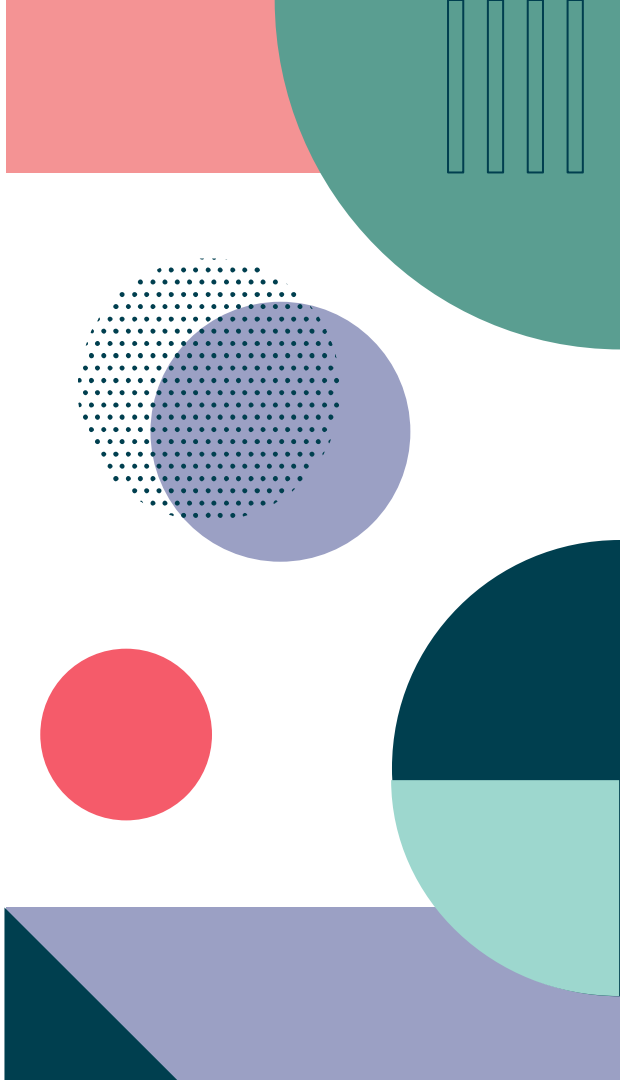


11 Self-organizing **teams**



12 Reflect and **adjust**





3

What is Agile

Iterative & Incremental Development

Iterative:

- Develop through repeated cycles
- Start simple, expected to change
- Used to find the right solution, fail early
- Used to improve the candidate solution

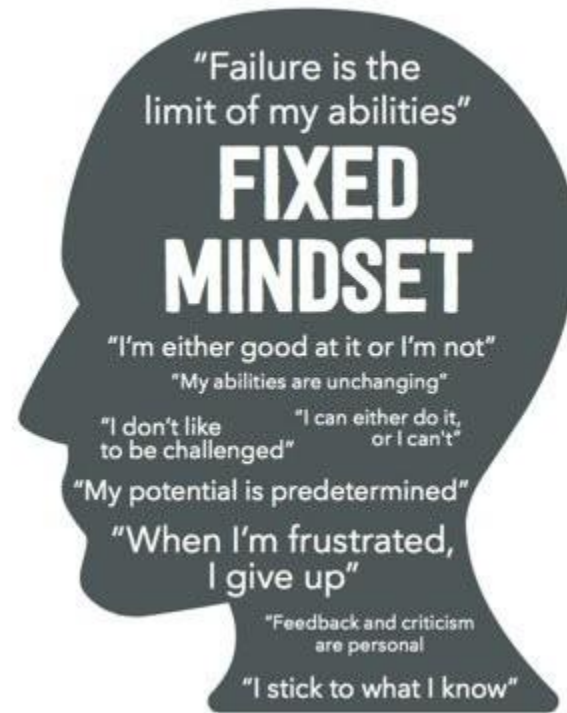


Incremental:

- Develop smaller portions at a time
- Gradually build up functionality
- Allows value to be delivered early

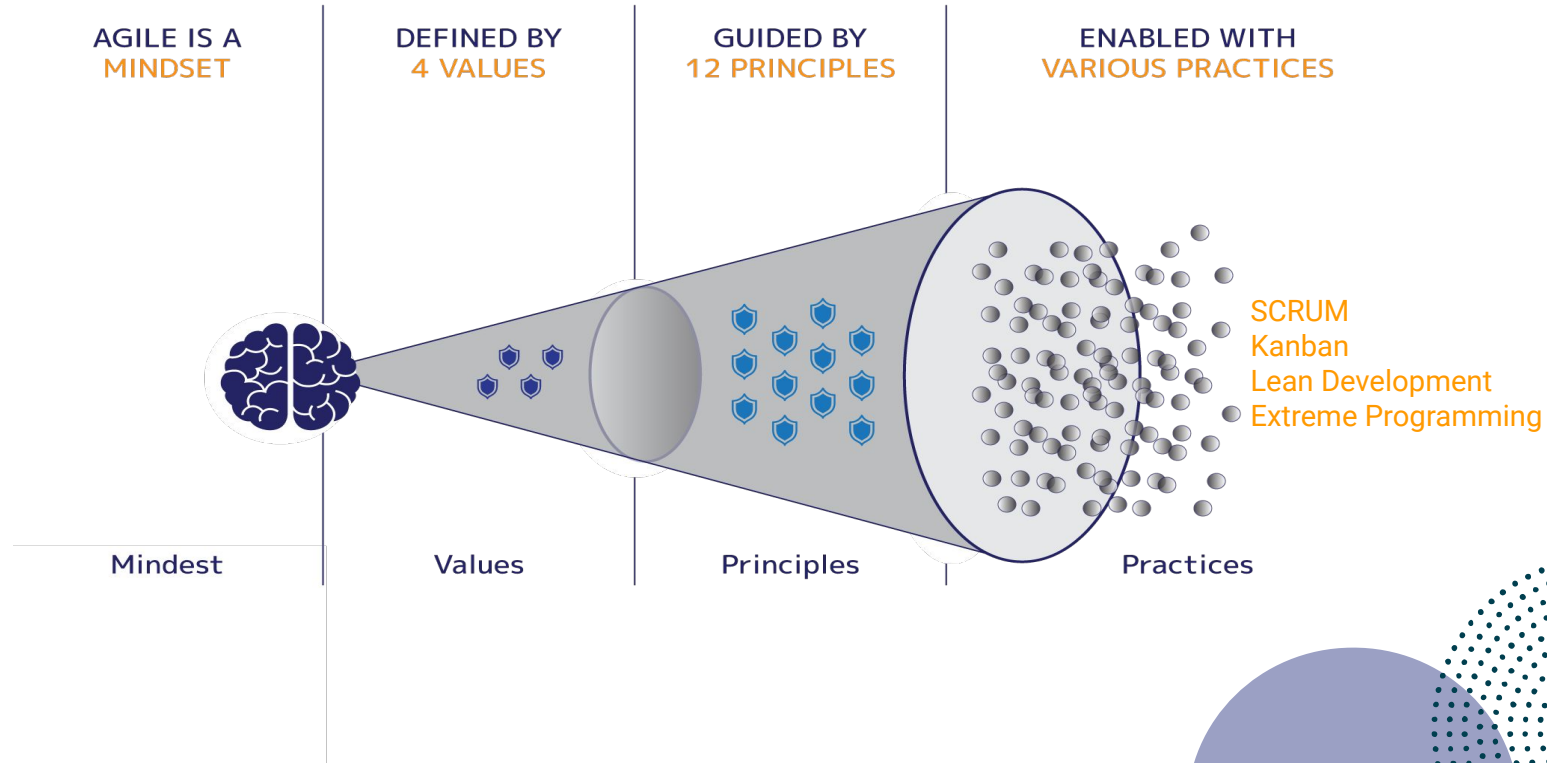


Growth vs Fixed Mindset



"Mindset," by Stanford psychologist Carol Dweck

What is Agile?



4

Lean



Lean - Toyota Production System (TPS)

In 1934, Toyota entered the automobile industry. Inspired by Henry Ford's workflow innovations and Frederick Winslow Taylor's book, Principles of Scientific Management, Toyota engineers Shigeo Shingo and Taiichi Ohno sought to eliminate waste by scaling back their production processes to the bare essentials, aka operating "lean."

An overall objective is to limit resources used in the manufacturing system to only those needed. There are six other objectives that are a key to obtaining the overall objective.

First — Optimize each individual step of the manufacturing system. In other words, make each part as efficient as possible to get the most from the least.

Second — Make a product with no flaws or defects. This ensures that each part of the production line will go as planned.

Third — Reduce the manufacturing cost. The cheaper it is to make the product the larger the profit for the company.

Fourth — Make a product that is demanded by consumers. If there is no demand then there is only money lost.

Fifth — There needs to be flexibility in the system. Things will not always go as planned and the system must be flexible enough that it can be modified easily.

Sixth — There needs to be a strong and reliable relationship between customers and suppliers. Since Just-In-Time manufacturing means there is virtually no extra stock or materials on hand, companies need to rely on each other to be reliable and on time.

Lean Management Pillars

1

Delivering Value to Customer

Customer value= the sum of all activities that a customer is willing to pay for, because these activities add value to a product or service, by changing its function or form.

2

Eliminating Waste

Eliminate anything and everything that does not add value from the perspective of your customer.

3

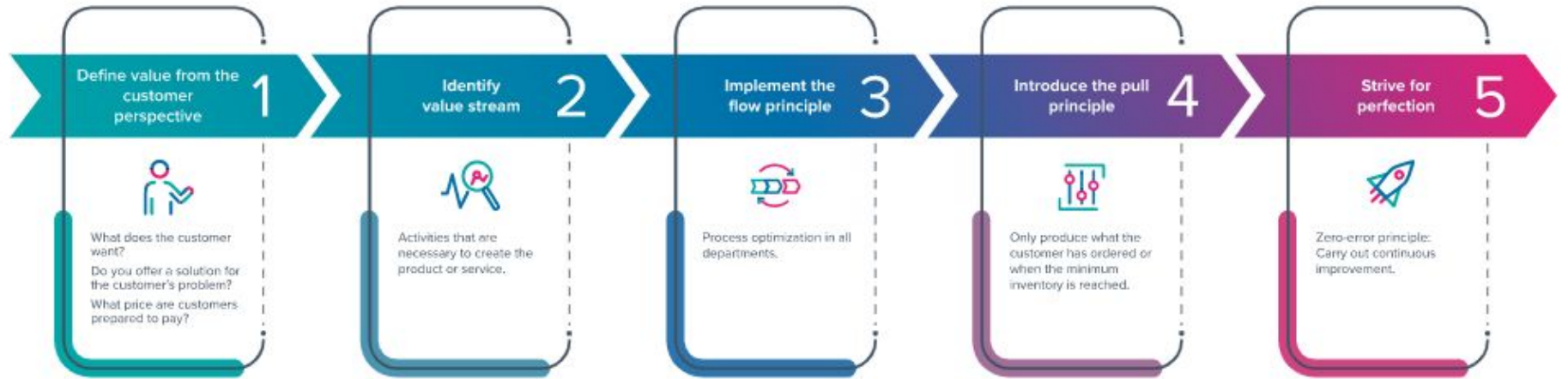
Continuous Improvement

constant experimentation, promoting knowledge sharing, and introducing small changes one at a time, you will cultivate a culture that "there is always a better way" to do things.

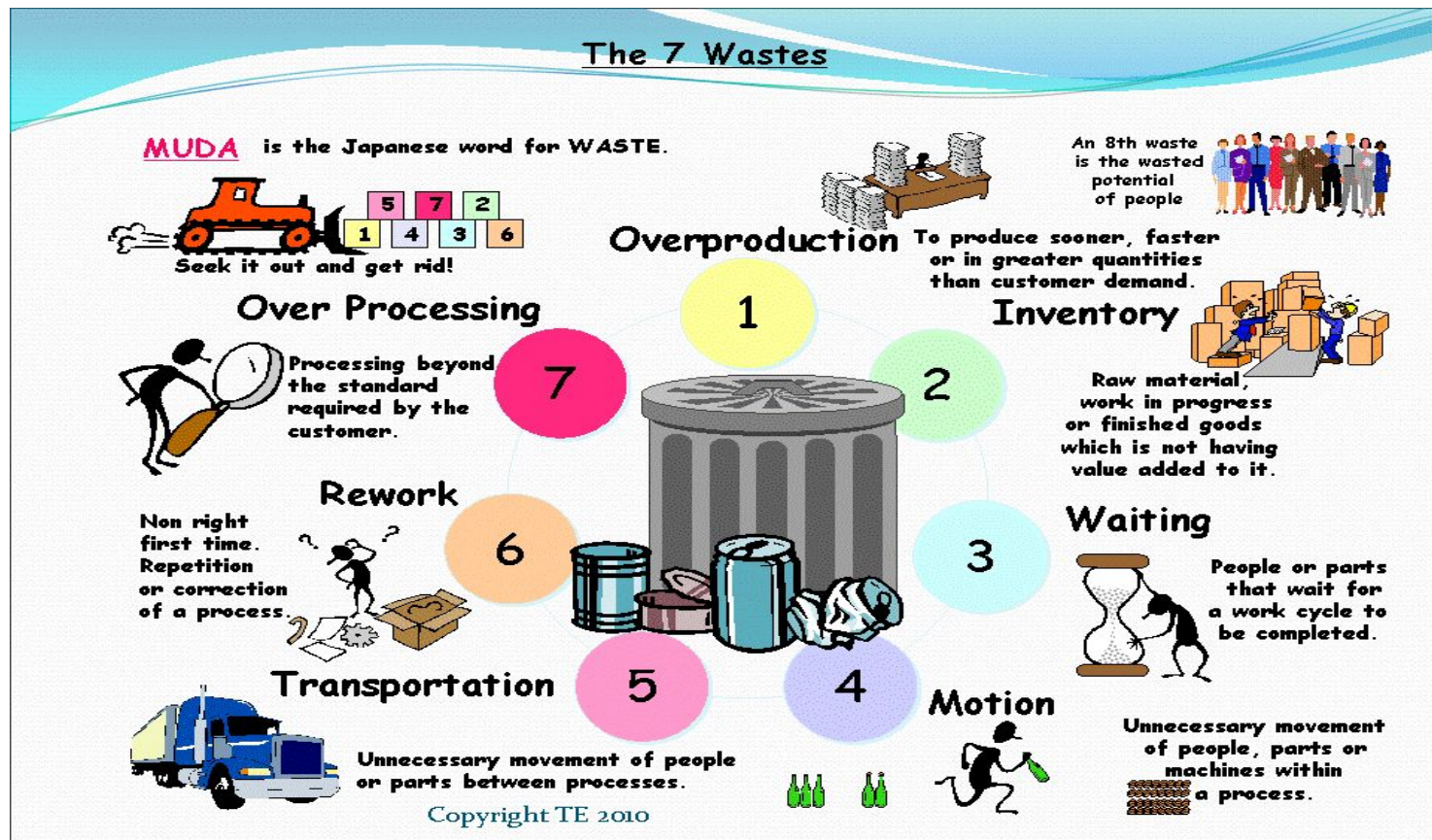
Lean management is an iterative project management technique that focuses on increasing a business's revenue by prioritizing the creation of valuable products for the customer, optimizing the use of the resources to eliminate waste, and establishing a system that keeps evolving along with the customer needs



Lean Principles



7 Types of Waste





Exercise: Let's define the waste

Let's think together:

- Let's pick 2-3 workflows you have been through in your life
- What are the wastes in each of these workflow?
- How can we eliminate this waste?



5

Kanban



Kanban

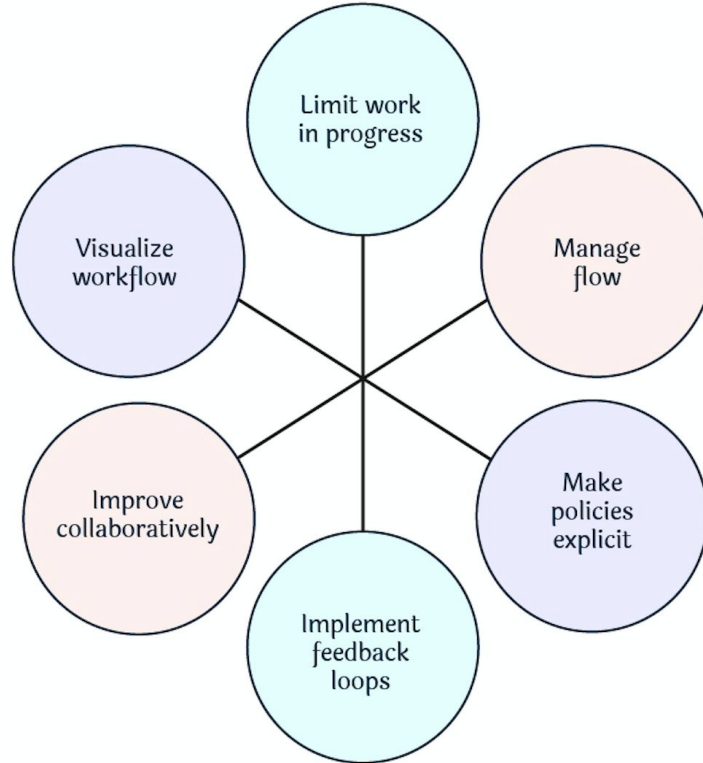
Japanese word “kanban”, meaning “visual board” or a “sign”

Pull System scheduling system

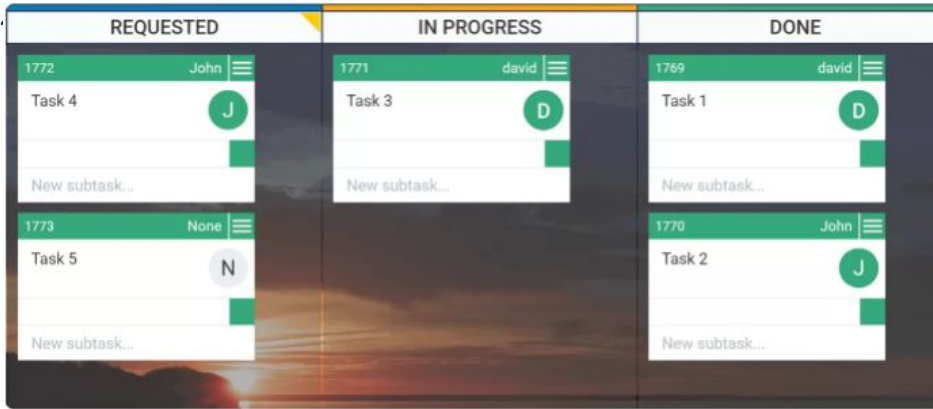
Change Management Principles	Service Delivery Principles
Start With What You Do Now	Focus on Customer's Needs and Expectations
Agree to Pursue Incremental, Evolutionary Change	Manage the Work, Not the Workers
Encourage Acts of Leadership at All Levels	Regularly Review the Network of Services



Kanban Practices



Kanban Practices (cont.)

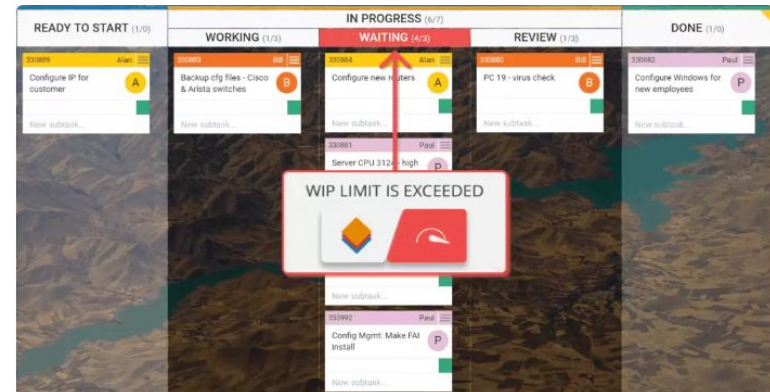


1. Visualize the Workflow

- A board with cards and columns.
- Each column on the board represents a step in the workflow.
- Each Kanban card represents a work item.
- The Kanban board itself represents the actual state of the workflow with all its risks and specifications.

2. Limit Work in Progress (WIP)

- Ensure a manageable number of active items are in progress at any one time
- Setting maximum items per stage ensures that a card is only “pulled” into the next step when there is available capacity.



Kanban Practices (cont.)

3. Manage Flow

- Managing the flow is about managing the work but not the people.
- By flow, we mean the movement of work items through the production process at a predictable and sustainable pace.

4. Improve Collaboratively (Using Models & the Scientific Method)

Cultivating an organizational culture where every hypothesis is proven to have positive or negative results is crucial for developing a mindset focused on improvement through evolutionary change.

5. Make Process Policies Explicit

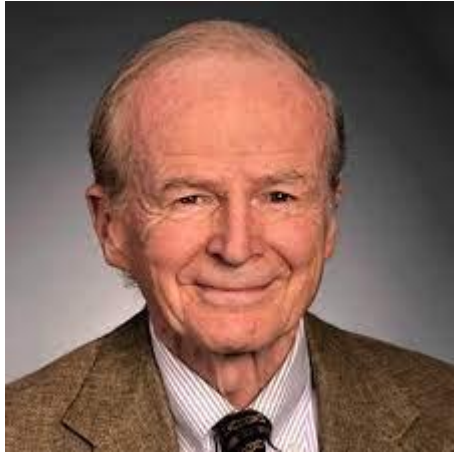
The process should be clearly defined, published, and socialized

6. Feedback Loops

- Kanban suggests the use of cadences (feedback loops) at a team level as well as service-oriented cadences.
- Team-level cadence is the daily Team Kanban Meeting for tracking the status and the flow of work.
- Service-oriented cadences such as operations, service delivery, and risk meetings, aim to synchronize and improve your delivery of service.



Little's Law



John Little
1961
Massachusetts
Institute of
Technology (MIT)

$$\text{Cycle Time} = \text{WIP} / \text{Throughput}$$

- Cycle time: the amount of time on average an item spends in the system. An item can be anything from user stories in a development pipeline, coffee orders at a cafe to patients in a hospital system
- WIP: work in process — the average number of items in the system at any given time
- Throughput: the rate at which items move through (or are processed by) the system on average



Little's Law (cont.)

Start less, to finish more.

If the goal is to complete work sooner, starting more work is the worst course of action to take.

Little's Law (cont.)

LITTLE'S LAW – IN STARBUCKS



Minutes until I get coffee = $\frac{\text{Number of people in line}}{\text{people served per minute}}$

$$\frac{11 \text{ in line}}{1 \text{ per minute}} = 11 \text{ minutes to get coffee}$$



In a stable system, items enter and exit the system at the same rate.
All items that enter the system, will also exit it (that is, none are lost within the system)

Exercise: Resource Utilization Trap



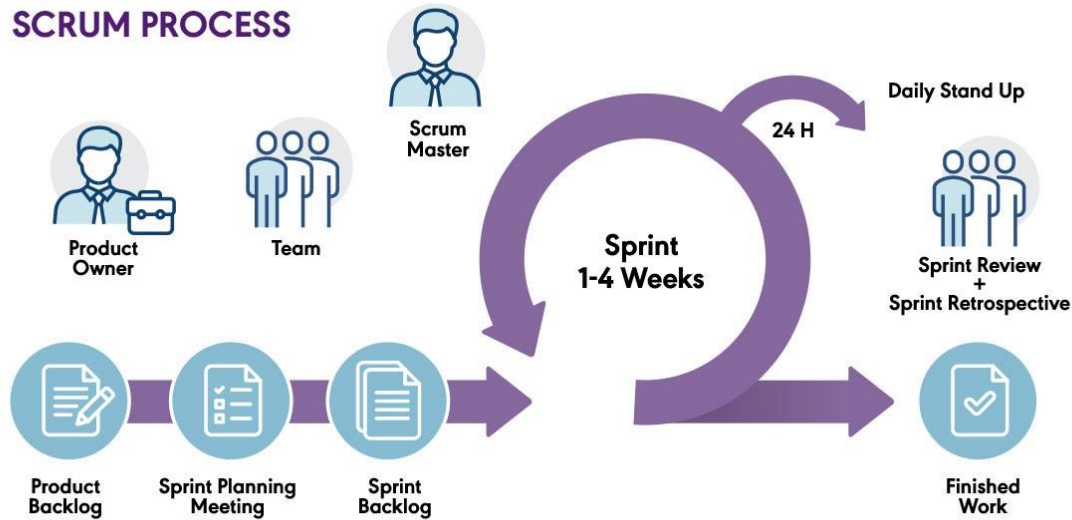
6

SCRUM



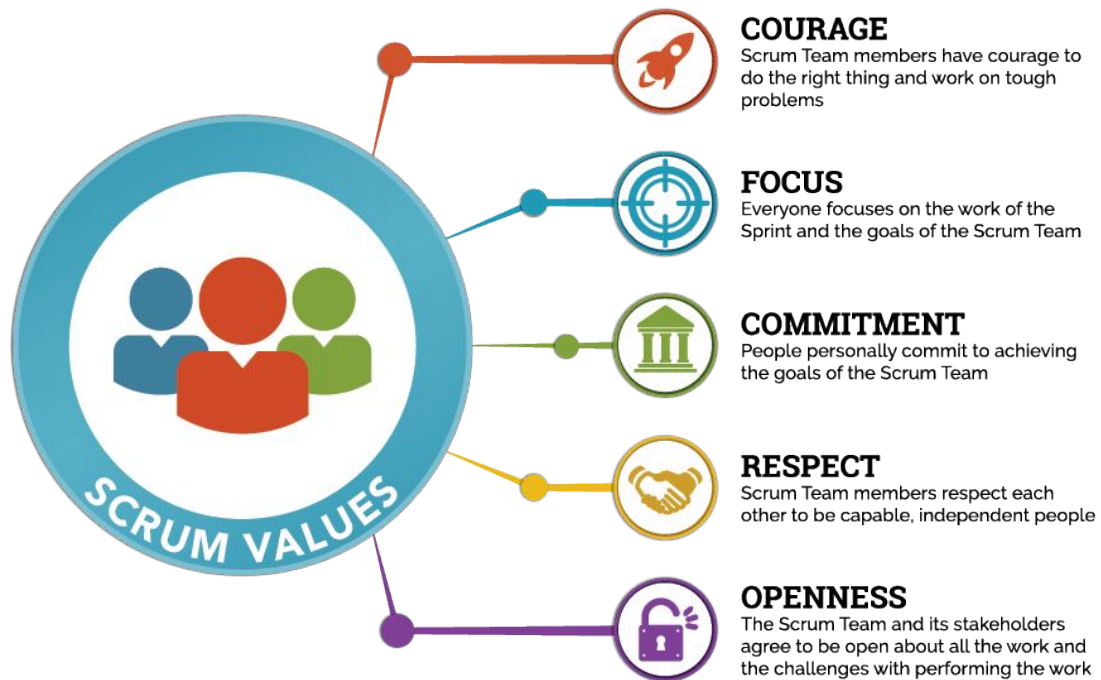
SCRUM

SCRUM PROCESS



Scrum helps people and teams deliver value incrementally in a collaborative manner

SCRUM Values



SCRUM Roles

Scrum Roles: A different way of thinking, a better way to drive success

Scrum roles differ from traditional project roles.

By collaborating, a Scrum team delivers more business value, faster.

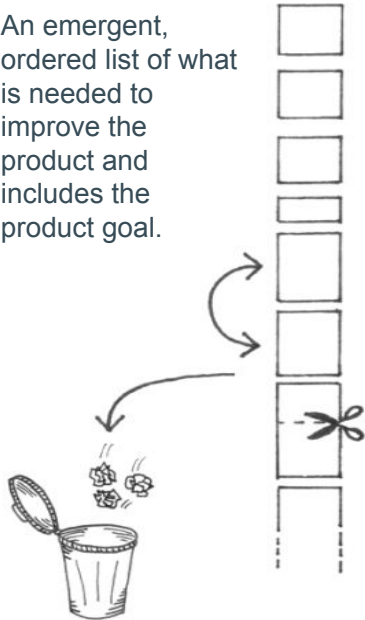


SCRUM Artifacts

PRODUCT GOAL

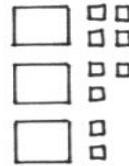
PRODUCT BACKLOG

An emergent, ordered list of what is needed to improve the product and includes the product goal.



SPRINT GOAL

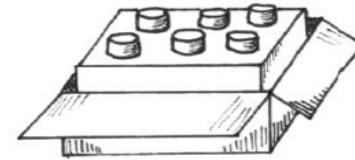
SPRINT BACKLOG



The set of product backlog items selected for the sprint by the developers (team members), plus a plan for delivering the increment and realizing the sprint goal.

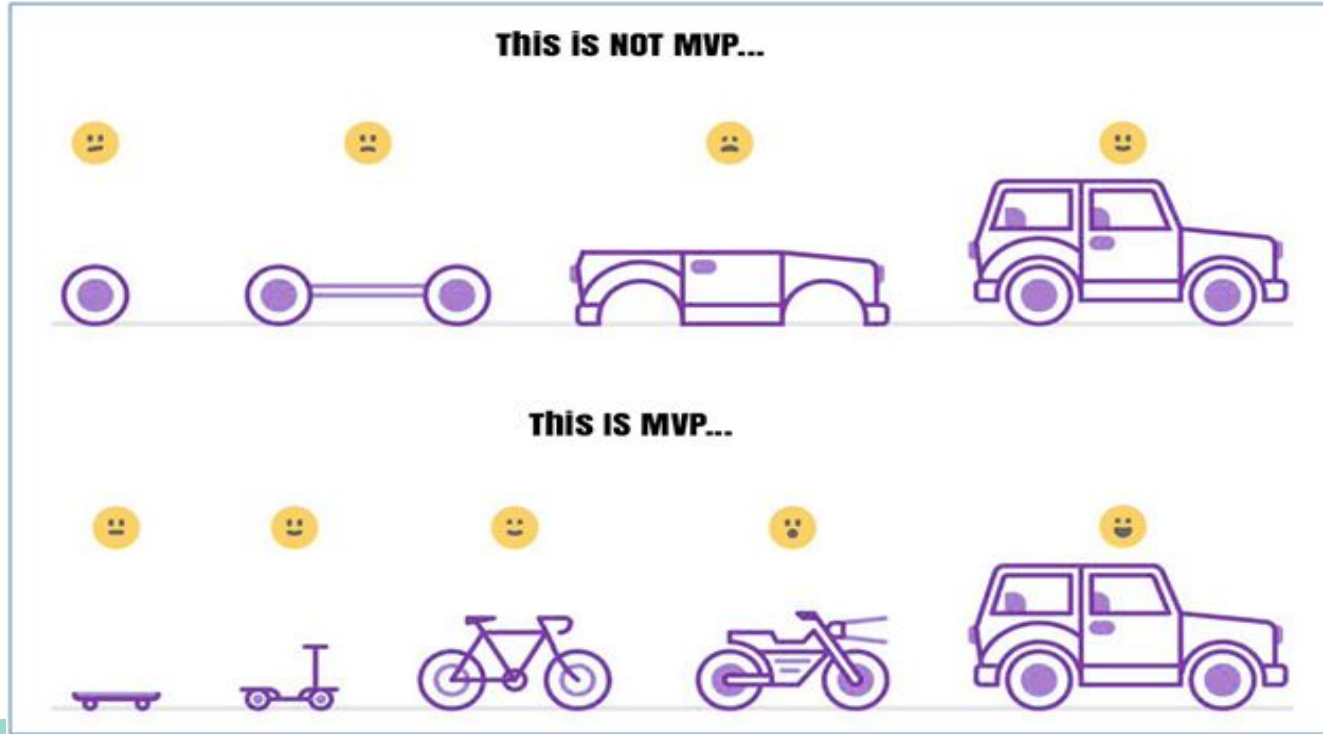
DEFINITION OF DONE

INCREMENT



A sum of usable sprint backlog items completed by the developers in the sprint that meets the definition of done, plus the value of all the increments that came before. Each increment is a recognizable, visibly improved, operating version of the product.

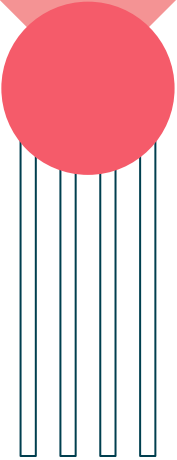
Minimum Viable Product



SCRUM Events



Refinement is a continuous activity used to add details, estimates, and order to items in the product backlog.



Sprint Goal

The **Sprint Goal** is an objective set for the Sprint that can be met through the implementation of Product Backlog.

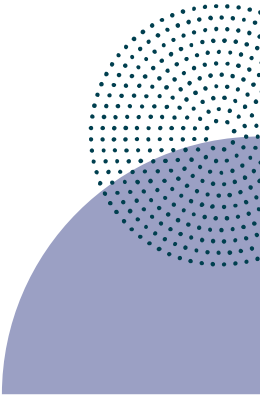
Sprint goals are the result of a negotiation between the Product Owner and the Development Team.

Sprint Goals should be specific and measurable.

Although the Sprint Goal is a commitment by the Developers, it provides flexibility in terms of the exact work needed to achieve it.

The Sprint Goal creates coherence and focus, encouraging the Scrum Team to work together rather than on separate initiatives.”

“The Sprint Goal is created during the Sprint Planning event and then added to the Sprint Backlog. As the Developers work during the Sprint, they keep the Sprint Goal in mind. If the work turns out to be different than they expected, they collaborate with the Product Owner to negotiate the scope of the Sprint Backlog within the Sprint without affecting the Sprint Goal.”



Daily SCRUM (Daily Standup)

Daily Scrum is to inspect progress toward the Sprint Goal and adapt the Sprint Backlog as necessary, adjusting the upcoming planned work.

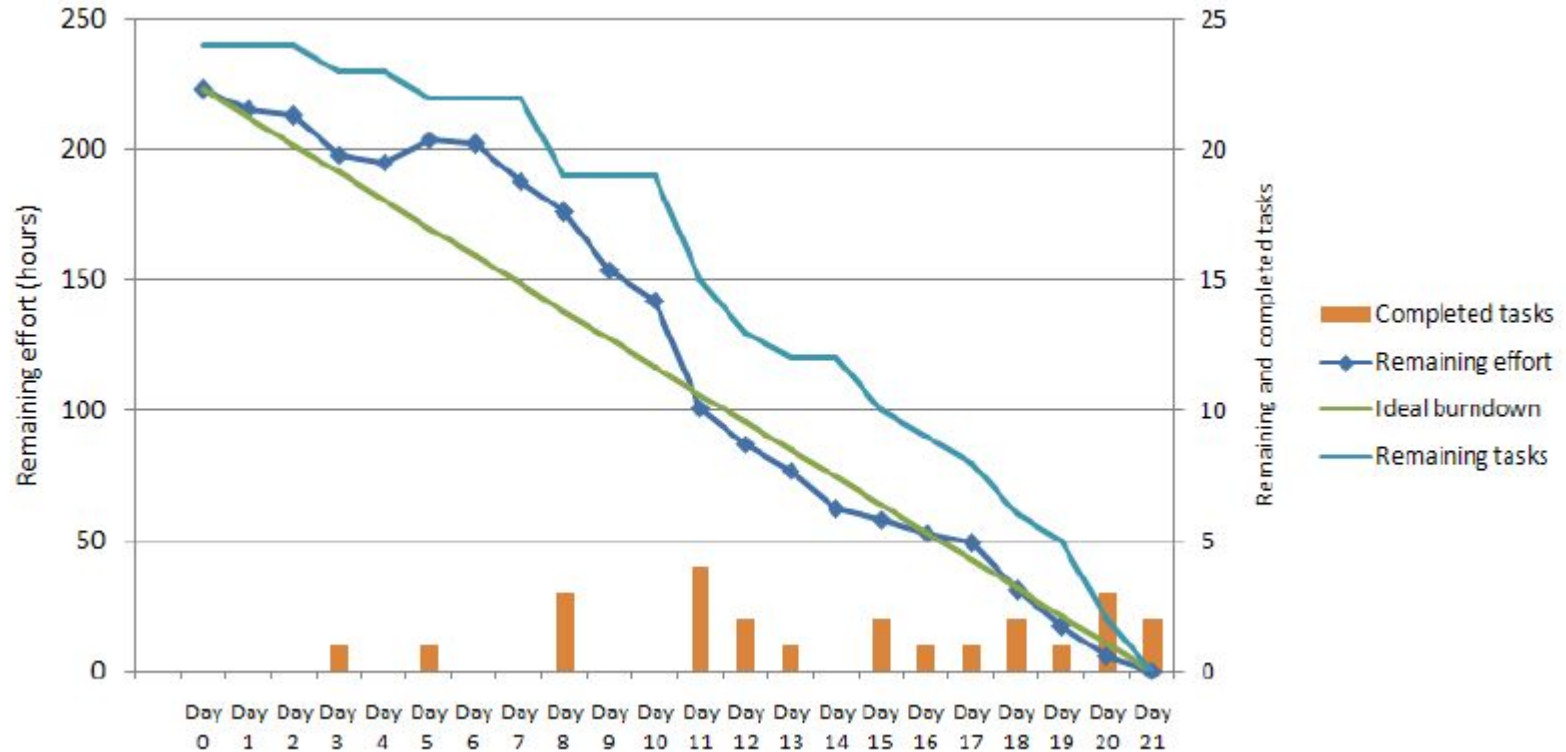
When & Where: The same time and place every working day of the Sprint.

Who attends: All the team, if PO or SM are working on sprint items, they participate as developers

The **Scrum Master** ensures that the meeting happens, but the Developers are responsible for conducting the Daily Scrum. The Scrum Master teaches them to keep the Daily Scrum within the 15-minute time-box.



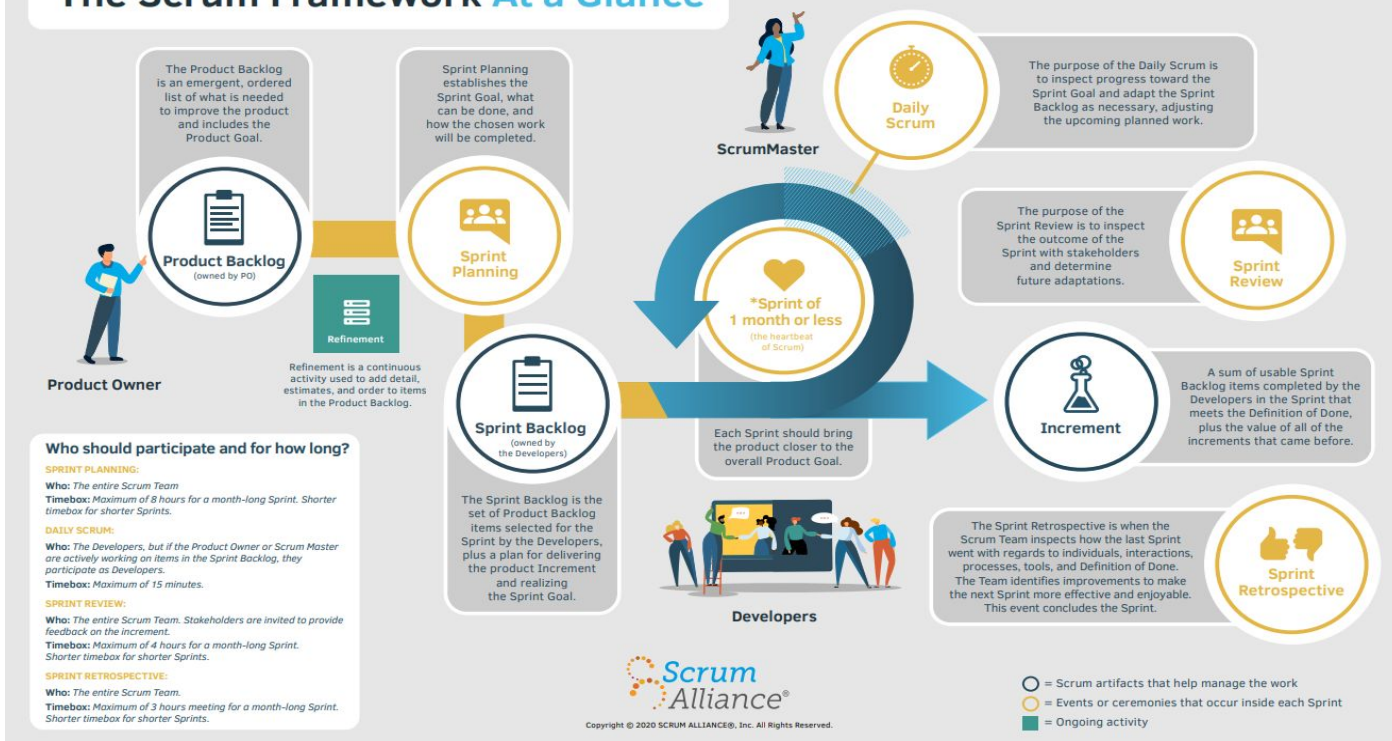
Sprint Burndown Chart



SCRUM

The Scrum Framework At a Glance

Version 5.0

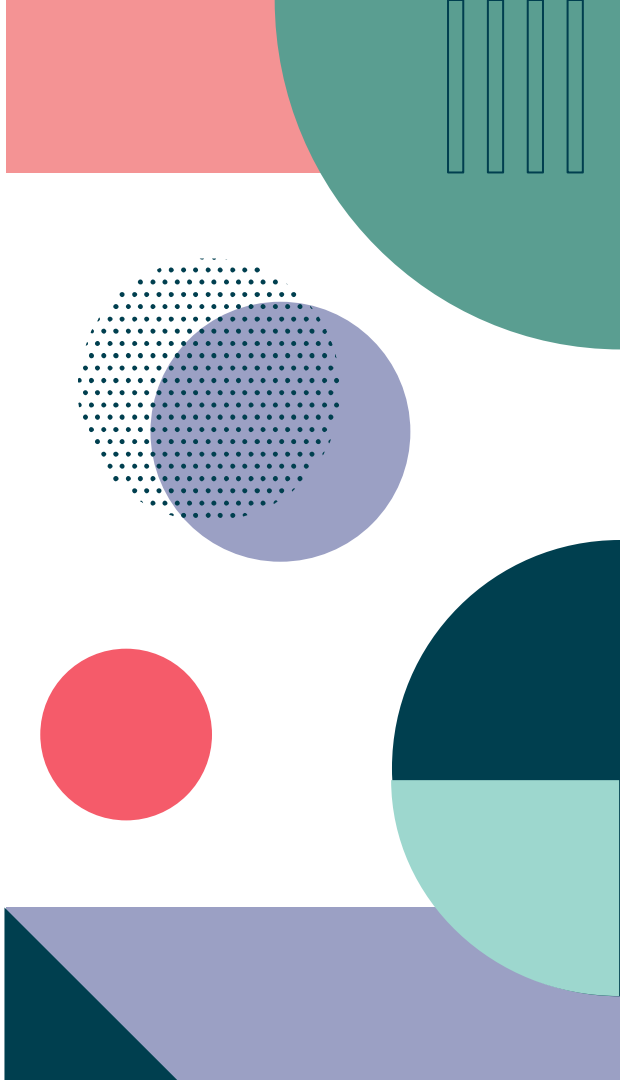


Exercise: Scrum Roles

Responsibility	Scrum Master	Product Owner	Development Team
Voice of the customer			
Documentation needed for Architecture Review			
Promote the Scrum Methodology			
Ensuring Goals, scope & product are understood by all			
Disseminating the vision and intent of the product			
Build and deliver the potentially shippable product increment			
Coaching the Development Team in self-organization and to work cross functionally			
Prioritize & Manage the product backlog			
Ensuring that the Product Backlog is transparent and understood by everyone			
Remove impediments			
Ensure Quality of the product			
Facilitating Scrum events as requested or needed			
Update system of record with status			
Define acceptance Criteria			
Manage stakeholders requirements			

Exercise: Scrum Roles (cont.)

Responsibility	Scrum Master	Product Owner	Development Team
Voice of the customer		✓	
Documentation needed for Architecture Review			✓
Promote the Scrum Methodology	✓		
Ensuring Goals, scope & product are understood by all	✓		
Disseminating the vision and intent of the product		✓	
Build and deliver the potentially shippable product increment			✓
Coaching the Development Team in self-organization and to work cross functionally	✓		
Prioritize & Manage the product backlog		✓	
Ensuring that the Product Backlog is transparent and understood by everyone		✓	
Remove impediments	✓		
Ensure Quality of the product			✓
Facilitating Scrum events as requested or needed	✓		
Update system of record with status			✓
Define acceptance Criteria		✓	
Manage stakeholders requirements		✓	



7

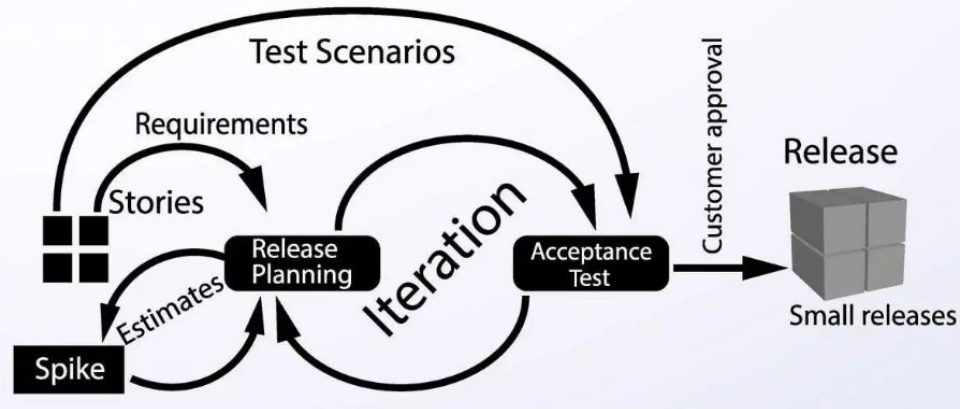
Extreme Programming XP

Extreme Programming

Extreme Programming (XP) is an agile software development framework that aims to produce higher quality software, and higher quality of life for the development team.

XP is the most specific of the agile frameworks regarding appropriate engineering practices for software development.

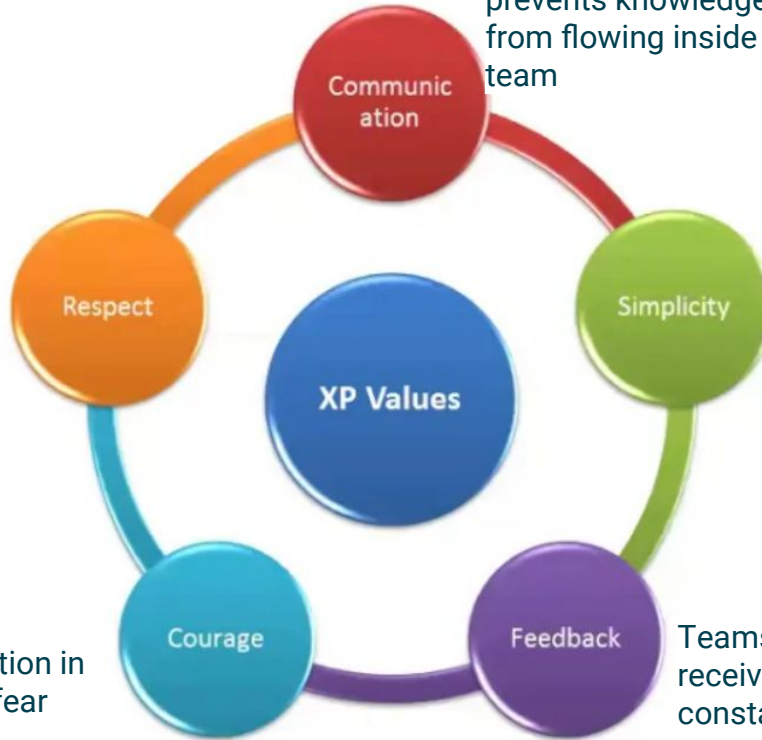
Extreme Programming



XP Values

Every person is
worthy of dignity
and respect

Effective action in
the face of fear



Lack of communication
prevents knowledge
from flowing inside a
team

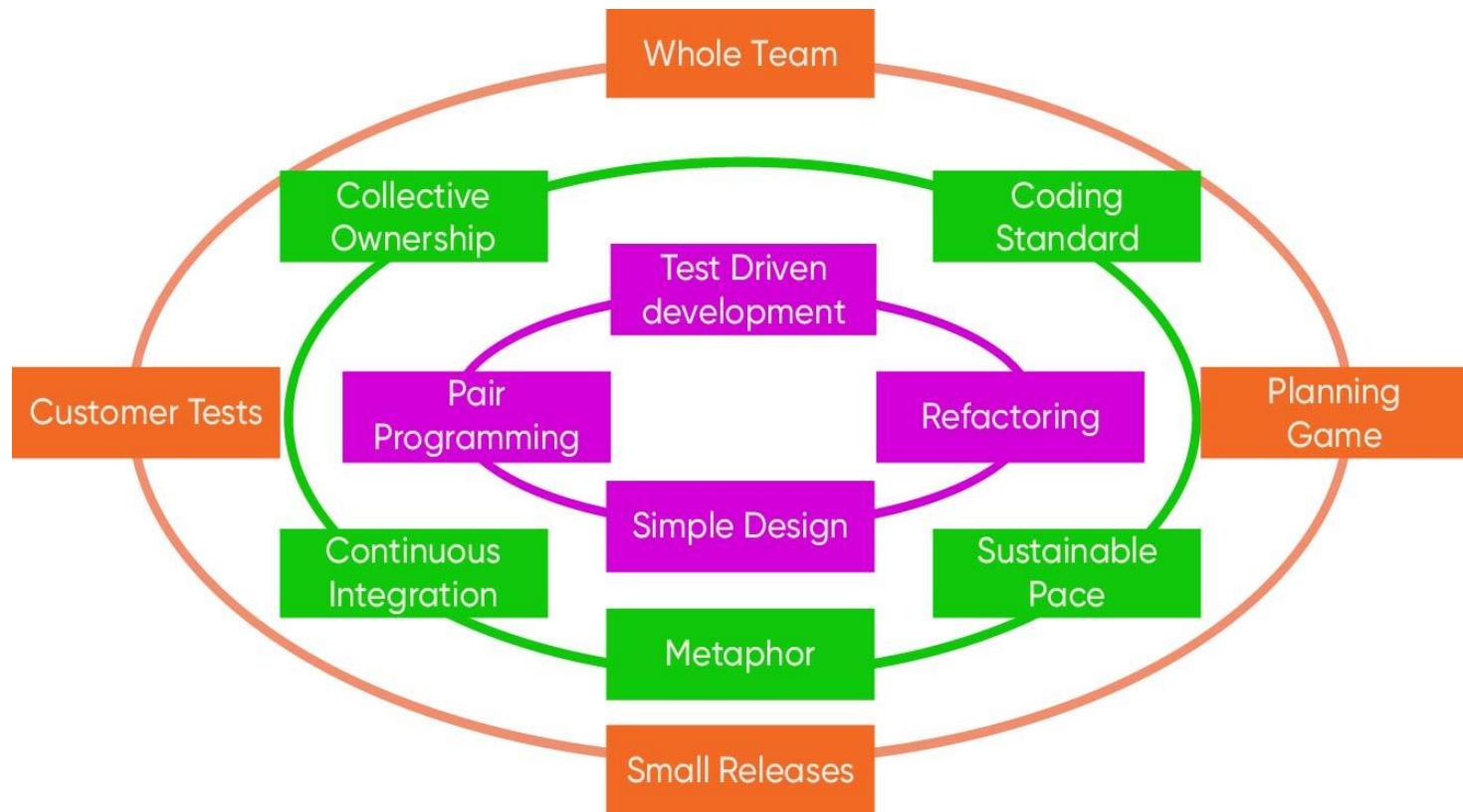
Always strive to do
the simplest thing
that works

Teams strive to
receive early,
constant feedback

XP Roles

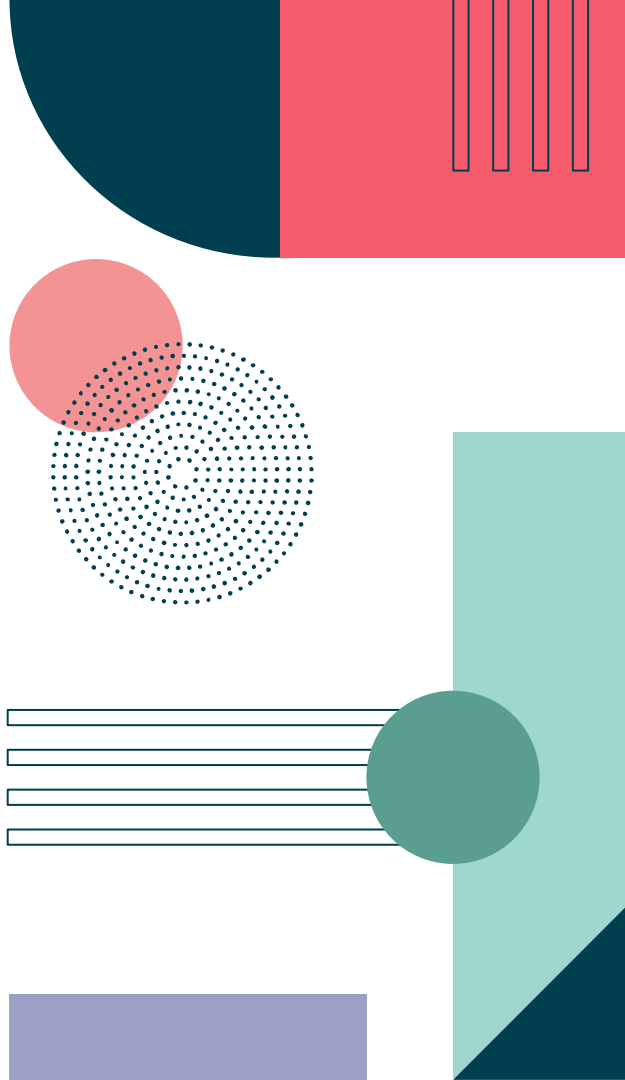
- **Customer:** ideally, there should be a real customer on-site to answer questions, prioritize user stories or collaborate with acceptance testing. When this isn't possible, this role could be fulfilled by a customer representative.
- **Programmers:** in an XP team, programmers estimate the effort needed to complete tasks and stories, write automated tests, and implement the stories.
- **Coach:** Having a coach isn't required, and many teams have achieved success without one. However, having someone experienced in XP to coach a team can ensure the team members follow the practices, turn them into habits and don't revert to the old ways.
- **Tracker:** A tracker tracks the team's progress metrics and talks with every team member to identify roadblocks and figure out solutions. The tracker calculates the metrics that indicate how well the team is doing, such as velocity and burndown charts, or the team uses a digital scrum or kanban board that calculates them automatically.

XP Practices





Teams



Self Organizing Teams

Traditional Teams



Agile Teams



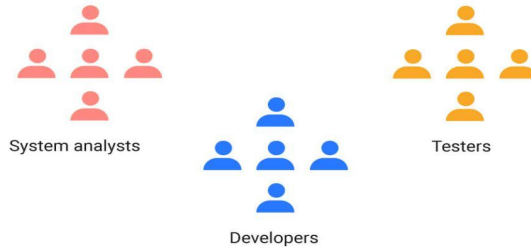
- Management Driven
- Workforce of isolated specialist
- Information is limited
- Segregated goals
- Focus on problem solving
- High Management Commitment
- Incremental improvement
- Management Controlled
- Policy/Procedure Based

- Customer Driven
- Multitasked Workforce
- Information Shared Widely
- Shared Goals
- Focus on achievements of objectives
- Higher worker commitment
- Continuous improvement
- Self controlled
- Values/Principal Based

Cross Functional Teams

Functional

Common functional expertise



Cross - Functional

Representatives from the various functions

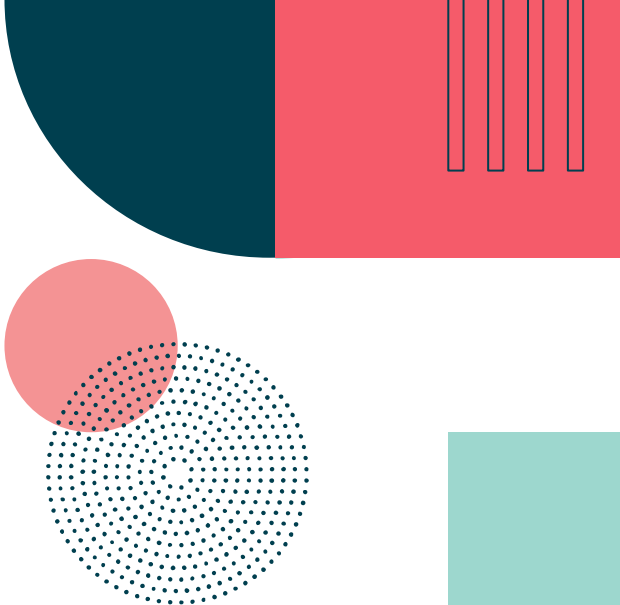


- Groups consisting of people from same functional areas
- Coordination is done through traditional ways
- Innovation is only done within the function
- Long cycle time due to moving back & forth between teams

- Groups consisting of people from different functional areas
- Improved coordination across functional areas
- Increased innovation in product and process
- Reduced cycle times for key customer touch points

Exercise: Self Organizing Teams





9

Definitions



Definition of Ready

Definition of Ready, the team makes explicit and visible the criteria (generally based on the INVEST matrix) that a user story must meet prior to being accepted into the upcoming iteration.

DOR Checklist
☐ User story has a business value
☐ estimated by the development team
☐ User story is clear and well-defined
☐ User story is achievable within a sprint
☐ User story is measurable and testable upon completion
☐ User story dependencies have been identified

Expected Benefits:

- Avoids beginning work on features that do not have clearly defined completion criteria, which usually translates into costly back-and-forth discussion or rework
- Provides the team with an explicit agreement allowing it to “push back” on accepting ill-defined features to work on

Definition of Done

The definition of done (DoD) is when all conditions, or acceptance criteria, that a software product must satisfy are met and ready to be accepted by a user, customer, team, or consuming system.

Release: DOD Checklist
☐ Code completion
☐ Release environment made ready
☐ Unit test checked and green
☐ The functional test checked and green
☐ QA done, issue resolved if any
☐ To do annotations - resolved
☐ All teams members approve work
☐ No work left pending in any stage is checked
☐ Continuous integration and TDD verified

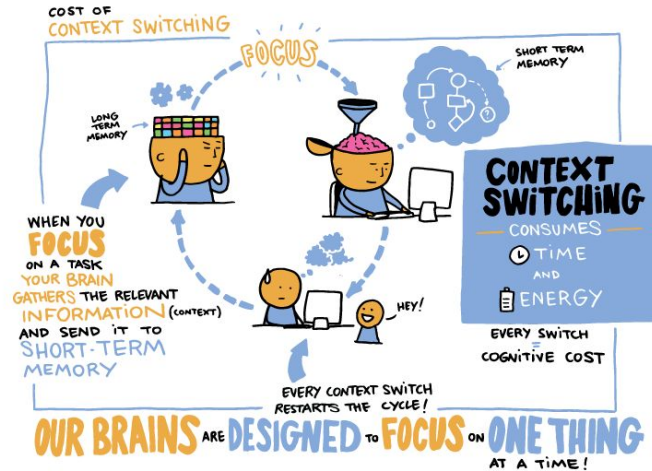
Expected Benefits:

- Provides a checklist which usefully guides pre-implementation activities: discussion, estimation, design
- Limits the cost of rework once a feature has been accepted as “done”
- Limits the risk of misunderstanding and conflict between the development team and the customer or product owner



90% DONE is NOT DONE

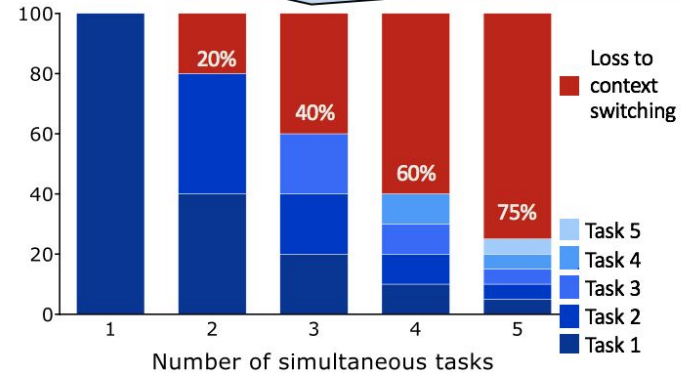
Task Switching



Fragmented Multi-tasking Kills Productivity

Percent of Time

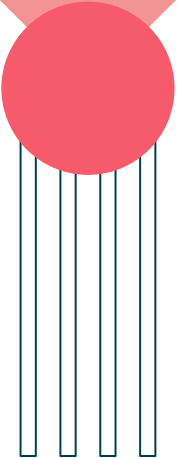
A 6 month task would take only 3.6 months without context loss



Source: Weinberg, Gerald M. (1992) Quality Software Management: Systems Thinking. Dorset House, p284

CLT

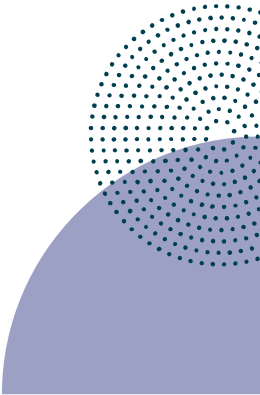
Copyright 2017 Collaborative Leadership Team



Exercise: Task Switching

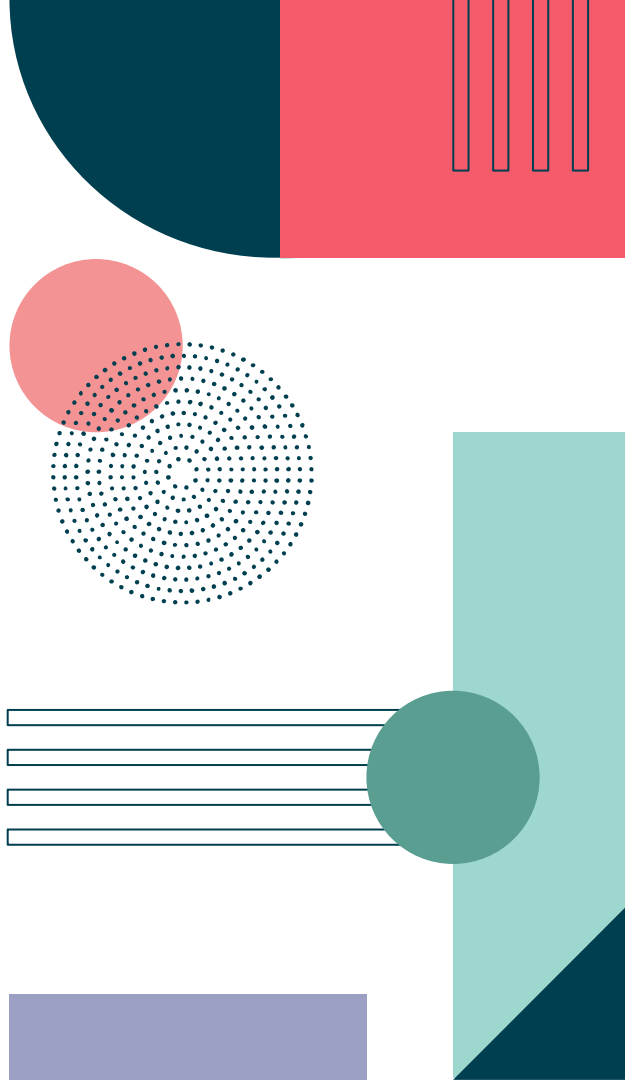


Let's Play:

- Divide yourselves into teams of 6
 - Define the writer person
 - Each one in the team should hold a stopwatch
 - The writer should write the name of each member twice, but the member himself/herself should spell the letter:
 - a. Writing first letter of each name alone for all names & repeating for following letters
 - b. Writing the whole name at once
 - Each member should stop the stop watch after the writer delivers his/her name and compare the 2 times together
- 

10

User Stories



User Stories

A **user story** is a short, simple description of a feature told from the perspective of the person who desires the new capability, usually a user or customer of the system.

As a <user role>

I want <goal>

so that <benefit>.

Front side of User Story card

User Story Name

As ____

I want to ____

So that ____

Value

Author

Estimate



Back side of User Story card

Success criteria

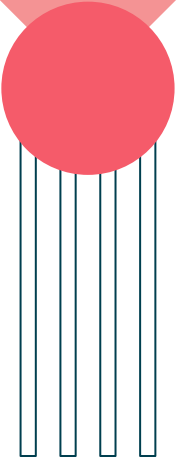
I know it is successful when ____

User Story Example

As a user I can login
so that I can access
my dashboard

- Username at least 6 characters
- Password must contain number and capital letter
- Lock out after 3 failed attempts

U. ...
PW. ...
→



Good User Stories



I ndependent

Independent stories can be freely re-ordered in the product backlog. Sometimes you can't get rid of an order dependency but it should be an exception.

Negotiable / Negotiated

A user story is the reminder to have a conversation. In that conversation the team negotiates the concrete solution, the "I want" part. The story may be enhanced or rewritten.

V aluable / Vertical

Each story adds something useful for the end user / customer - the "so that" part. This leads to vertical increments: E.g. a working slice of front end, scripts & DB, instead of a finished DB without front end.

E stimable

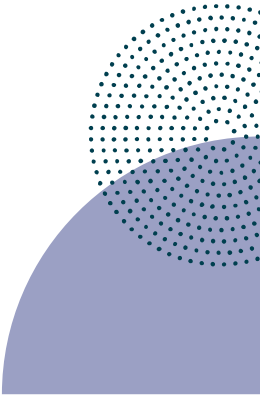
You need a rough effort estimate to guestimate ROI and order the backlog. If you can't estimate, you need to a) break the story into pieces or b) better understand what value it's meant to add or c) explore unknown tech in a time-boxed research spike.

S mall

Small stories are easier to estimate and test and hide fewer misunderstandings. "Small" can be 1 day in a web shop or 3 person-weeks for a medical product. At the very least, the team must be able to finish a story ("done done") in 1 iteration.

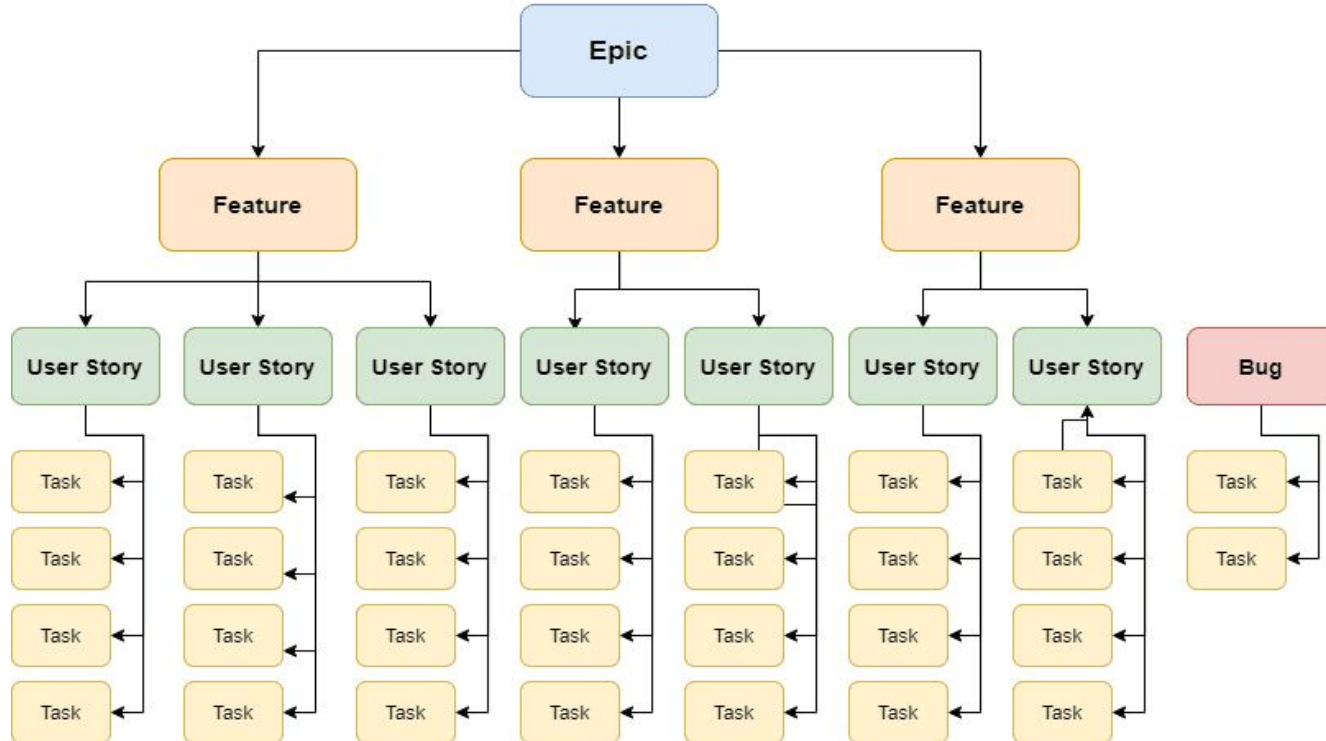
T estable

It must be possible to write a test (at least in theory) for each story. Otherwise, how will you confirm that the story is done? Sometimes test cases are given as acceptance criteria. If you can't think of a test, the story is probably to fuzzy.



User Stories

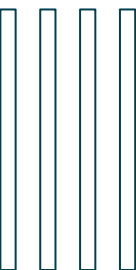
PRODUCT BACKLOG





Who & When?

Who writes user stories

- The Product Owner is accountable for the product backlog
 - Anyone can write the user stories including team members or even the customer
 - All the team including development team with the PO and/or customer should discuss user stories together
- 

When to write user stories

- User stories are written at all development stages: before development, while shaping a product idea
 - During development, when a developer finds out that the story is actually an epic (large user story),
 - At the release stage (there could be unexpected difficulties during the transition).
- 

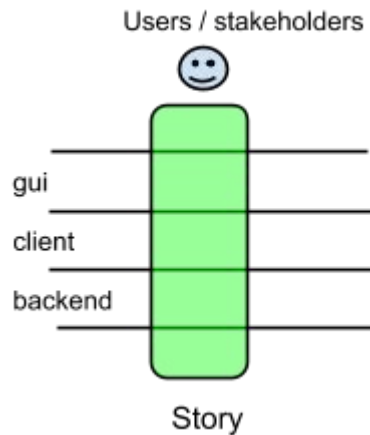
Slicing

Slicing allows you to deliver meaningful value to users

Vertical Slicing allows you to cut across multiple architectural layers to deliver meaningful experience/value to your users.

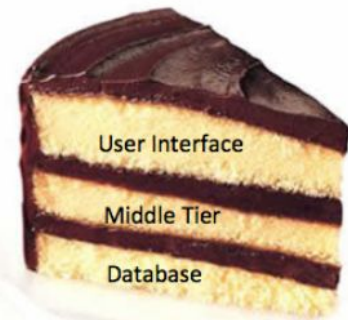
It's very different to the traditional approach of trying to deliver the UI only approach which doesn't deliver real value to users.

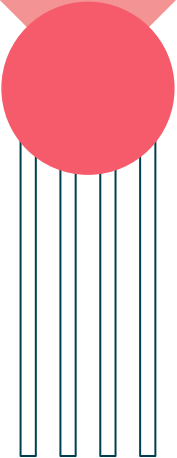
Story = vertical, testable, user valuable.



Slicing Criteria

1. INVEST
2. Think MVP
3. Breaking down stories by acceptance criteria
4. Breaking down by Happy/Unhappy Path
5. Breaking down by test scenario/test cases
6. Breaking down by users personas
7. Quality Attributes

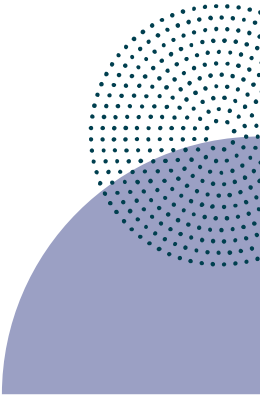


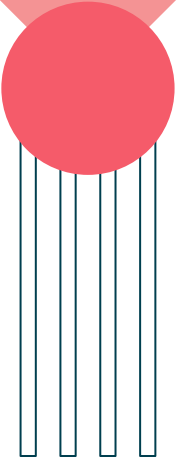


Prioritization

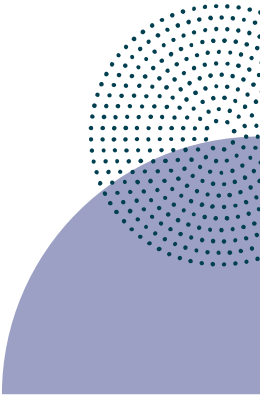
Prioritization in literary terms means the decision of arranging things in order of their importance. Prioritization in agile is the act of deciding in what order the agile team will work on the requirements in a project.

Prioritization Techniques:

1. MoSCoW prioritization – popularized by the DSDM methodology.
 2. Kano model – introduced by Prof. Noriaki Kano
 3. The relative weighting method – by Karl Wieggers
 4. Opportunity Scoring
 5. Stack Ranking
 6. Priority Poker
 7. Cost of Delay
 8. 100 Dollar Test
- 
- 

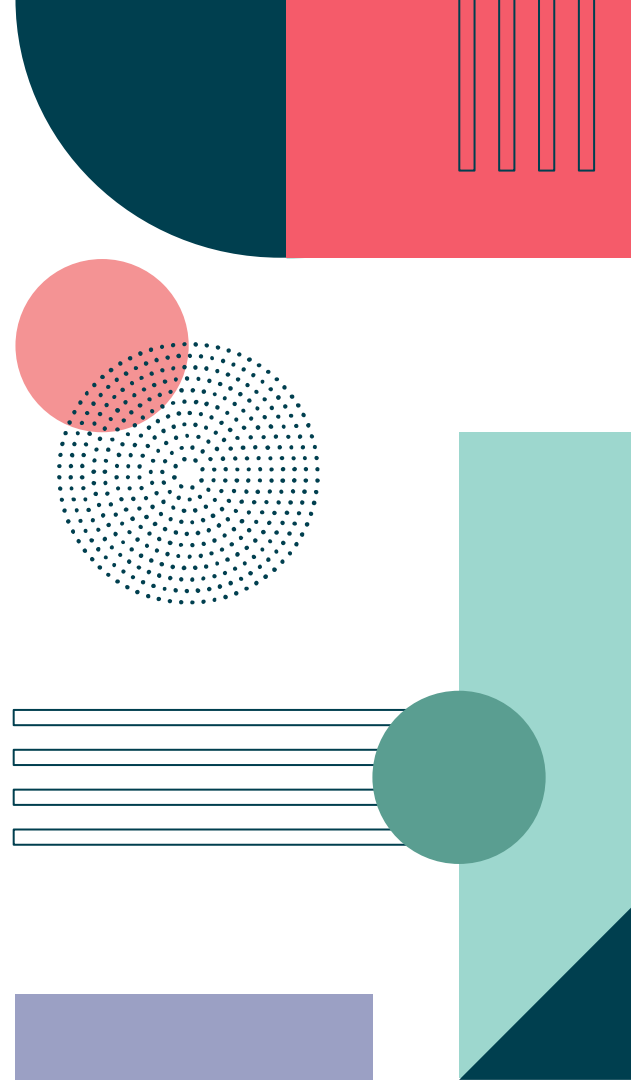


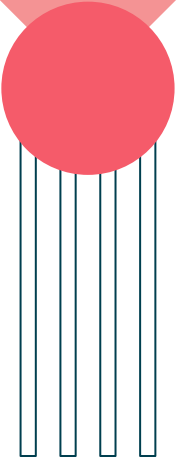
Exercise: Let's create our product backlog

- Imagine your day since you wake till you sleep to be your roadmap
 - Divide your roadmaps to EPICs
 - Pick 1 or 2 EPICs and divide them to stories
 - Write the stories in the correct format with the correct acceptance criteria as you are implementing a customer requirement
- 
- 

11

Estimation





Estimation

Agile estimation is the project management mechanism used to derive an idea about the costs and effort involved in executing a project. It helps in deciding whether the work can be undertaken as proposed, needs to be modified or should be cancelled. .

Agile estimates are essential for

- Making teams accountable for deliverables,
 - Inducing discipline across the Agile team,
 - Predicting the approximate time, it will take to finish a project,
 - Enabling better sprint management,
 - Improving team productivity.
- 



Levels of Agile Estimations



1. **Project or Proposal level**

Quick Function Point Analysis during the initial phases of the project development.

2. **Release Level**

Assigning story points to the user stories that can help in defining the order of the user stories by priority.

3. **Sprint Level**

Divide user stories into tasks and assign estimated hours to the tasks, based on their complexity. At this level, we also define the person who is responsible for the task along with the status of the tasks.



Story Points

Story points are units of measure for expressing an estimate of the overall effort required to fully implement a product backlog item or any other piece of work. Teams assign story points relative to work complexity, the amount of work, and risk or uncertainty.

Why use Story points?

- Dates don't account for the non-project related work
- Dates have an emotional attachment to them.
- Each team will estimate work on a slightly different scale, which means their velocity (measured in points) will naturally be different. This, in turn, makes it impossible to play politics using velocity as a weapon.
- Once you agree on the relative effort of each story point value, points can be assigned quickly without much debate.
- Story points reward team members for solving problems based on difficulty, not time spent. This keeps team members focused on shipping value, not spending time.



T-Shirt Size Estimation

T-shirt sizing is a project estimation and **capacity planning tool** that helps you track how much time or effort an initiative will take.

1. Start by deciding on your sizes
2. Align on what each size represents
3. Decide who assigns t-shirt sizes
4. Assign t-shirt sizes to each initiative
5. Track t-shirt sizes using a work management tool
6. Use t-shirt sizing to gauge workload



shutterstock.com · 585194636

Planning Poker Estimation

Planning Poker is a consensus-based estimating technique. Agile teams around the world use Planning Poker to estimate their product backlogs. Planning Poker can be used with story points, ideal days, or any other estimating unit.

1. The product owner or customer reads an agile user story or describes a feature to the estimators.
2. Each estimator is holding a deck of Planning Poker cards with values like 0, 1, 2, 3, 5, 8, 13, 20, 40 and 100.
3. The estimators discuss the feature, asking questions of the product owner as needed.
4. Each estimator privately selects one card to represent his or her estimate. All cards are then revealed at the same time.
5. If all estimators selected the same value, that becomes the estimate.
6. If not, the estimators discuss their estimates. The high and low estimators should especially share their reasons.
7. After further discussion, each estimator reselects an estimate card, and all cards are again revealed at the same time.
8. The poker planning process is repeated until consensus is achieved or until the estimators decide that agile estimating and planning of a particular item needs to be deferred until additional information can be acquired.



Exercise: Relative Sizing



Skate board



Bus



Car



Scooter



Tricycle



Cycle



Aeroplane



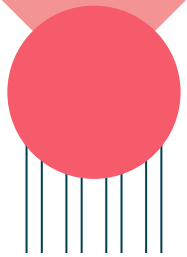
Space Shuttle



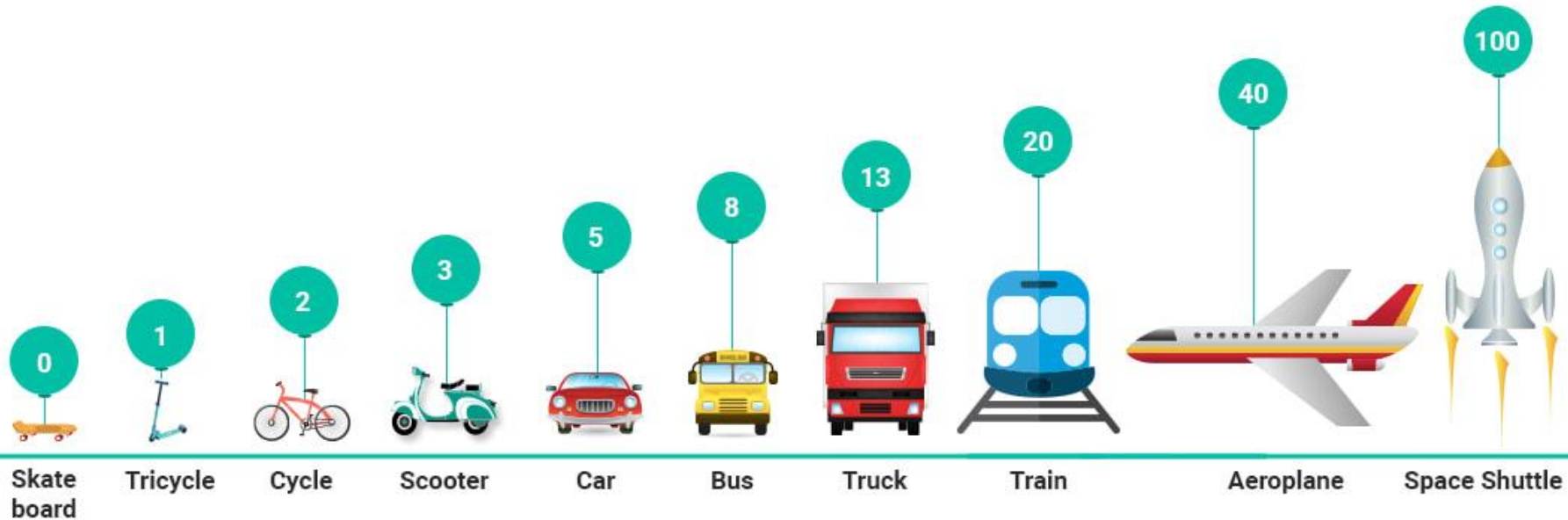
Train



Truck

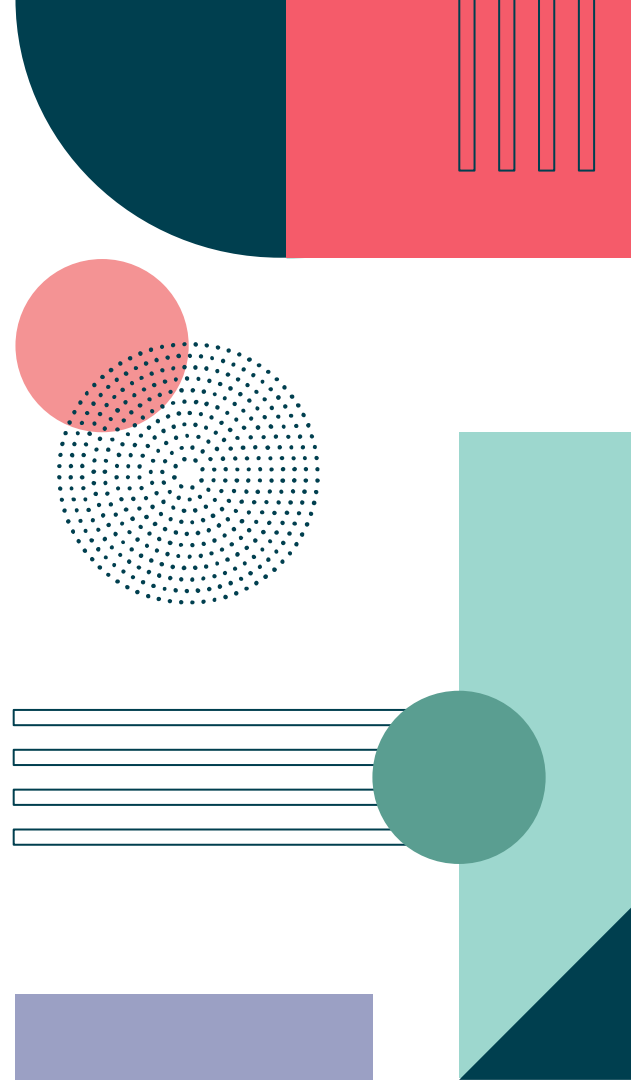


Exercise: Relative Sizing (cont.)



12

Retrospectives



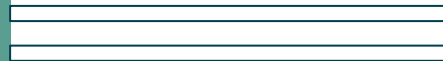


Retrospectives

At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

(Agile Manifesto)

Reflection is a mean by which practitioners can develop a greater level of self-awareness about the nature and impact of their performance, an awareness that creates opportunities for professional growth and development



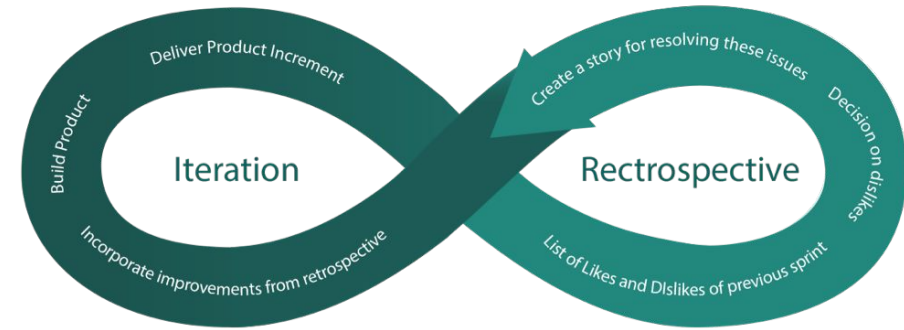
Retrospectives in Agile

Agile teams commonly incorporate reflection by holding a retrospective.

A retrospective is a meeting that's held at the end of an iteration in Agile software development.

The whole team attends the retrospective meeting, where they inspect how the iteration (sprint) has been done, and decide what and how they want to adapt their processes to improve.

A set of prioritized & defined actions coming out of a retrospective are communicated and done in the next iteration. That makes retrospectives an effective way to do short cycled improvement.



Types of Retrospectives

4 Ls: Liked, Learned, Lacked, Longed for

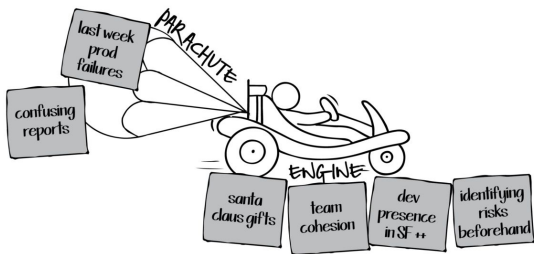
When we want to gather data on both positives and negatives at the end of a sprint.

Liked: things you really liked

Learned: things you have learned

Lacked: things you have seen the team doing but consider that could be done better

Longed for: something you desired or wished for

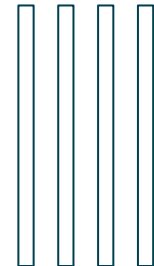


Engine: What has been pushing us forward? Making us move faster?

Parachute: What has been slowing us down?

Speed Car

Forward-thinking exercise. It is a mix of retrospective and futurespective



Small Starfish

Activity to foster the thinking around practices and the value the team get from it. It helps team members to understand each other perceived value on such practices.

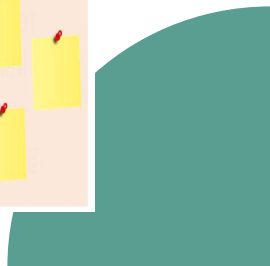
Start, Stop & Continue

Action-oriented retrospective style, generating an immediate list of practical ideas for continuous improvement

Start: Things a team member thing should add to the process

Stop: things someone thinks it's inefficient or is wasting time

Continue: things a member wants to continue to emphasize but are not yet a habit



Types of Retrospectives (cont.)

Mad, Sad & Glad

To highlight the positive feelings your team might have after a Sprint, but also to underline concerns or questions they might have going forward.

Mad: events that triggered frustration

Sad: events that triggered disappointments

Glad: events that caused positive feelings



GLAD
Things that have worked very well and that have made the team happy

Sprint 1	Sprint 2
Sprint 3	Sprint 4
Sprint 5	Sprint 6

SAD
Disappointments, things that have not worked as well as was hoped

Sprint 1	Sprint 2
Sprint 3	Sprint 4
Sprint 5	Sprint 6

MAD
Things that didn't work and/or have wasted a lot of time

Sprint 1	Sprint 2
Sprint 3	Sprint 4
Sprint 5	Sprint 6



One word to describe the sprint

One word Retrospective

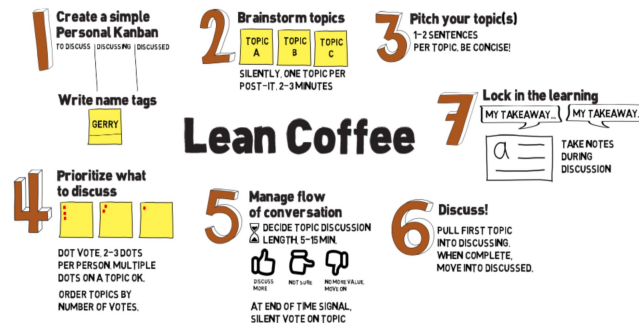
simple check-in activity that allows participants to share their feelings before getting into the data

Types of Retrospectives (cont.)

Lean Coffee

Conversations are directed and productive because the agenda for the meeting was democratically generated

Open Ended Conversations
owned & directed by team
Team make sure to emphasize
all ideas they have & want to
work on



PRODUCED BY GERRY KIRK WWW.GERRYKIRK.NET

A Typical Sprint Retrospective Model

What worked well?

What could be improved?

What will we commit to doing in the next Sprint?

Scrum Team members make actionable commitments

What went well: want to keep doing these things

What could be improved: went ok, but could be better

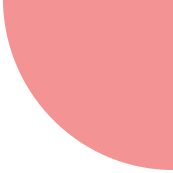
What went bad: want to stop doing these things

A focus on the next period: goals to focus on according to priority

Typical Retrospective



Suggested Agenda



01

Five minutes

Ice Breaker

02

Five minutes

Review Previous outstanding actions

03

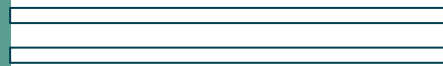
45 minutes

Retrospective Activity

04

Five minutes

Assign action items

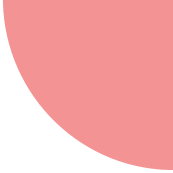


Smart Goals





Let's do our Retrospective



What went right?

What went wrong?

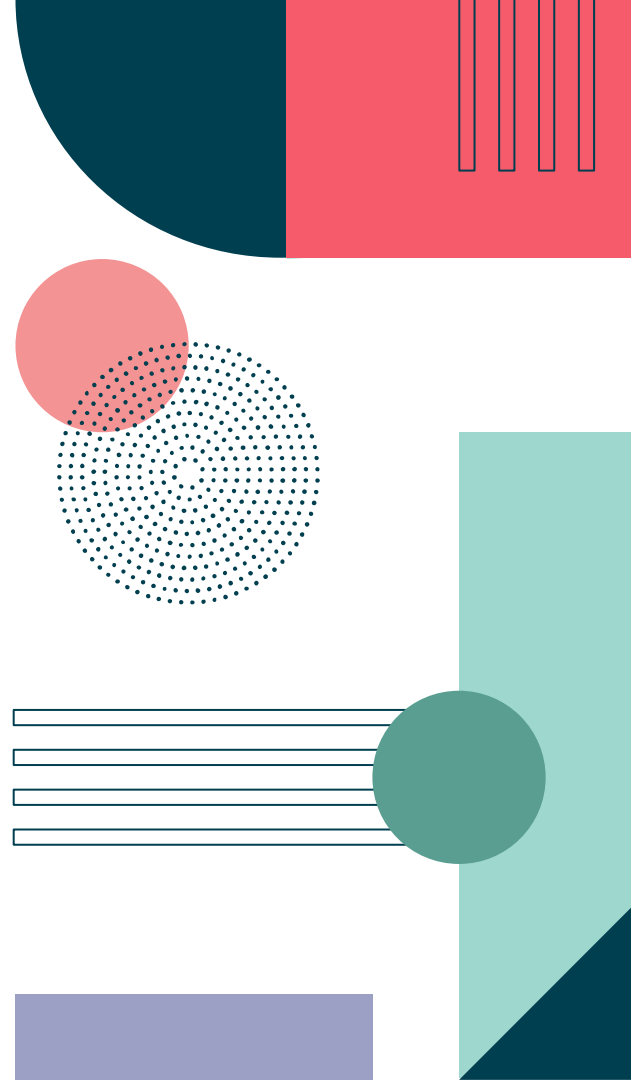
What to stop doing?

What to start doing?



13

Technical Practices



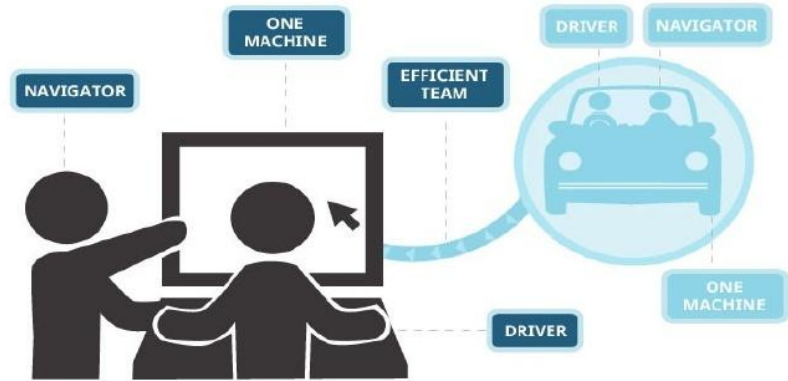
Pair Programming

Pair programming consists of two programmers sharing a single workstation (one screen, keyboard and mouse among the pair).

The programmer at the keyboard is usually called the “driver”, the other, also actively involved in the programming task but focusing more on overall direction is the “navigator”; it is expected that the programmers swap roles every few minutes or so.

Expected Benefits

- Increased code quality
- Better diffusion of knowledge among the team
- Better transfer of skills
- Large reduction in coordination efforts
- Improved resiliency of a pair to interruptions

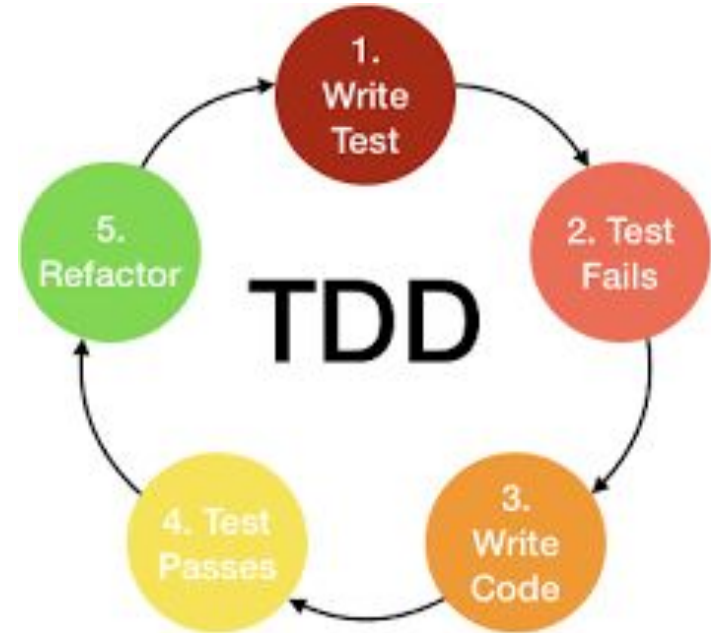


Test Driven Development (TDD)

A style of programming in which three activities are tightly interwoven: coding, testing (in the form of writing unit tests) and design (in the form of refactoring).

Expected Benefits

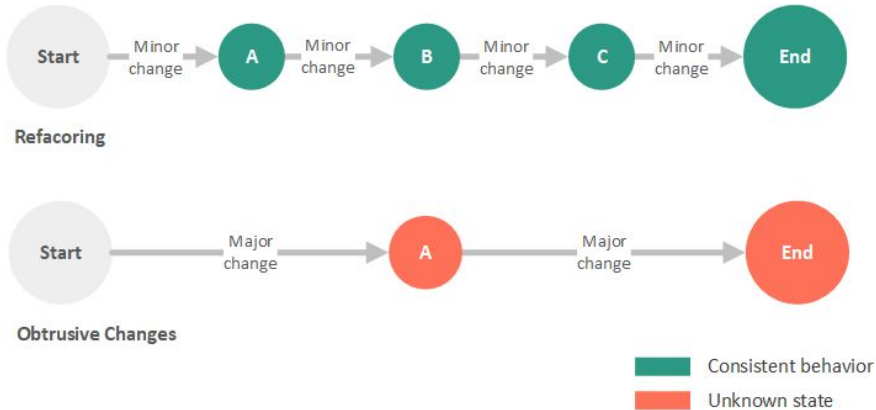
- Significant reductions in defect rates, at the cost of a moderate increase in initial development effort
- Overheads are more than offset by a reduction in effort in projects' final phases
- TDD leads to improved design qualities in the code, and more generally a higher degree of "internal" or technical quality, for instance improving the metrics of cohesion and coupling



TDD Steps

Refactoring

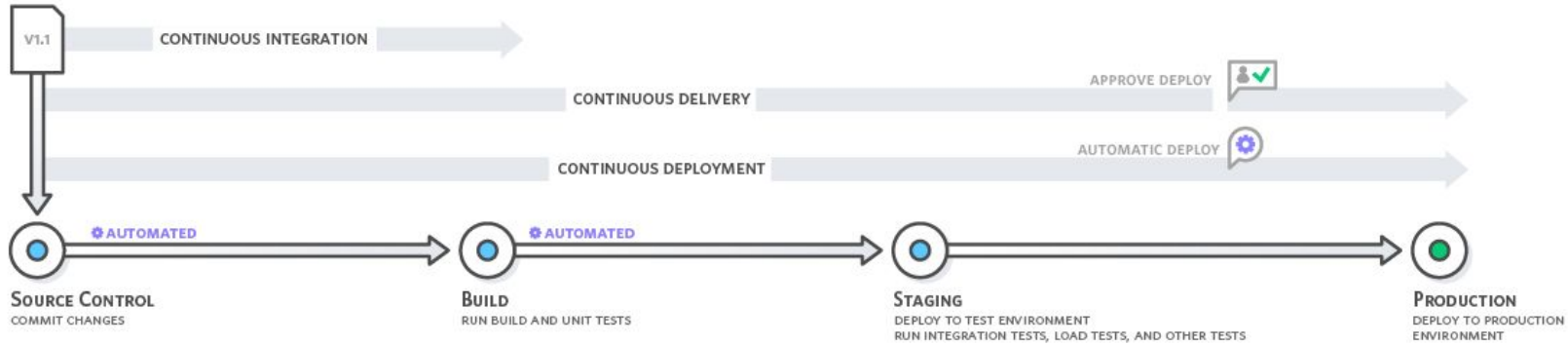
Refactoring is a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behavior.



Expected Benefits

- Improves objective attributes of code (length, duplication, coupling and cohesion, cyclomatic complexity) that correlate with ease of maintenance
- Helps code understanding
- Encourages each developer to think about and understand design decisions, in particular in the context of collective ownership / collective code ownership
- Favors the emergence of reusable design elements (such as design patterns) and code modules

CI/CD - Continuous Integration Continuous Deployment/Delivery



Continuous integration refers to the build and unit testing stages of the software release process. Every revision that is committed triggers an automated build and test.

Continuous delivery, code changes are automatically built, tested, and prepared for a release to production. Continuous delivery expands upon continuous integration by deploying all code changes to a testing environment and/or a production environment after the build stage.

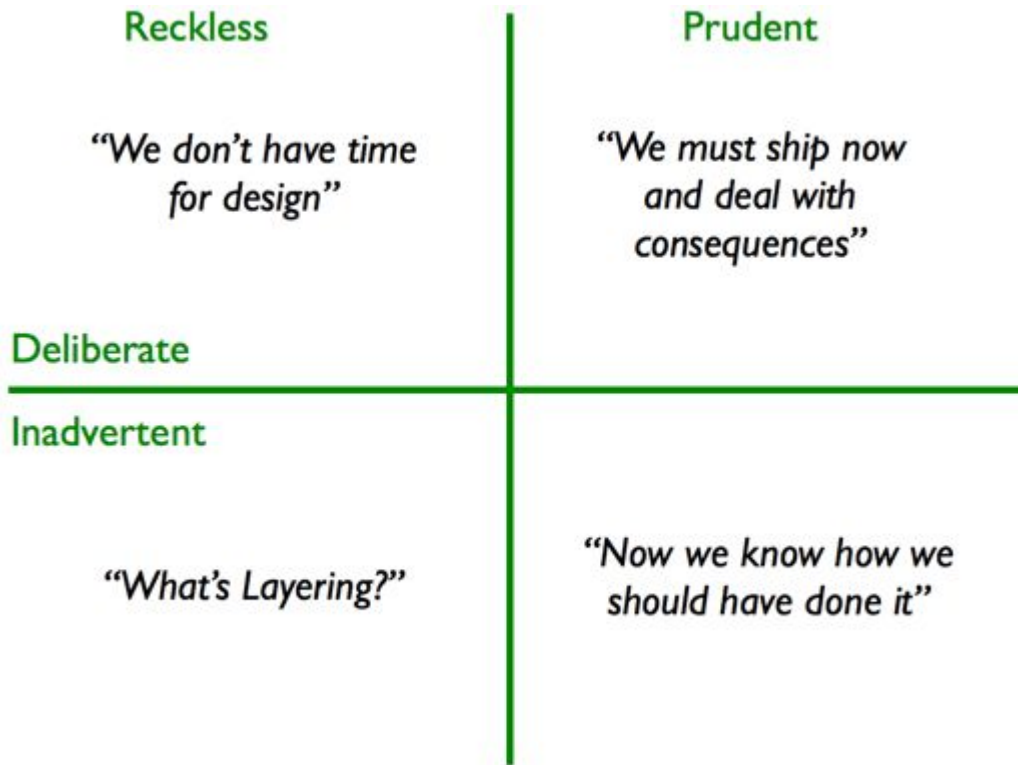
Expected Benefits

- Improve Developer Productivity, by freeing developers from manual tasks and encouraging behaviors that help reduce the number of errors and bugs released to customers.
- Find and Address Bugs Quicker, With more frequent testing, team can discover and address bugs earlier before they grow into larger problems later.
- Deliver Updates Faster, team deliver updates to their customers faster and more frequently

Technical Debt

Technical debt (also known as tech debt or code debt) describes what results when development teams take actions to expedite the delivery of a piece of functionality or a project which later needs to be refactored. In other words, it's the result of prioritizing speedy delivery over perfect code.

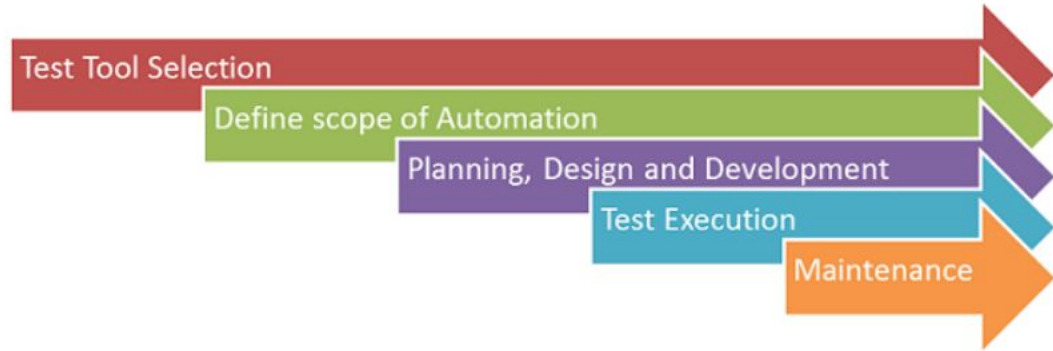
Managing technical debt requires a balance between quality and speed. Quick workarounds might mean you meet your deadlines, but make sure you know the cost. Technical debt may look harmless, but if left unchecked, you'll find that at some point, speed and agility are no longer an option.



Types of Technical Debt

Automated Testing

Automated testing is the application of software tools to automate a human-driven manual process of reviewing and validating a software product.



Process for Automation Testing

Test cases to be automated selection criteria:

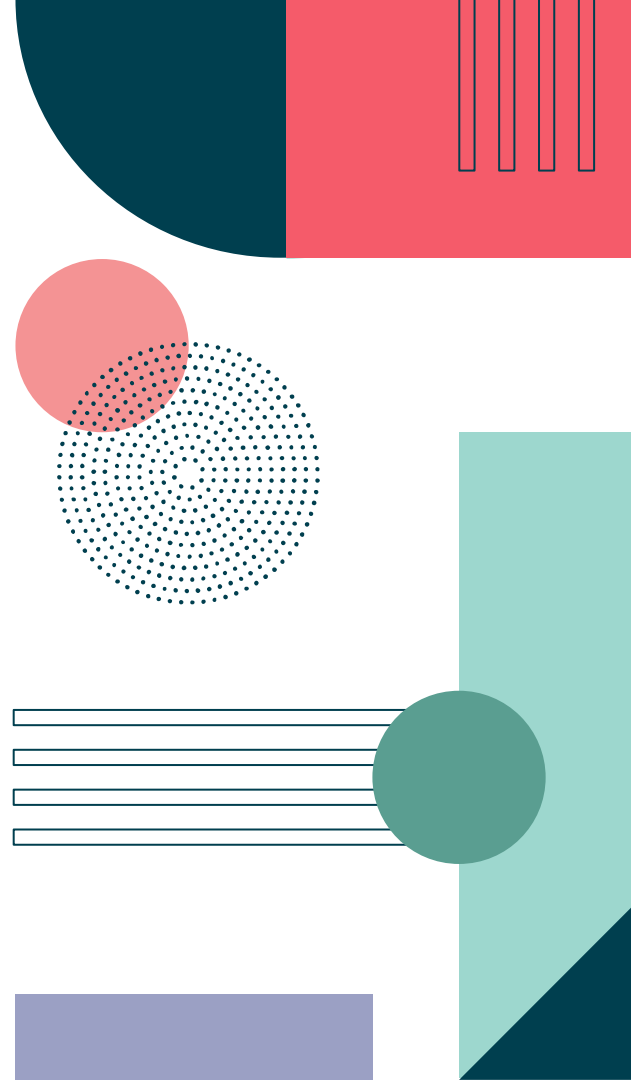
- High Risk – Business Critical test cases
- Test cases that are repeatedly executed
- Test Cases that are very tedious or difficult to perform manually
- Test Cases which are time-consuming

Test cases are not suitable for automation:

- Test Cases that are newly designed and not executed manually at least once
- Test Cases for which the requirements are frequently changing
- Test cases which are executed on an ad-hoc basis.

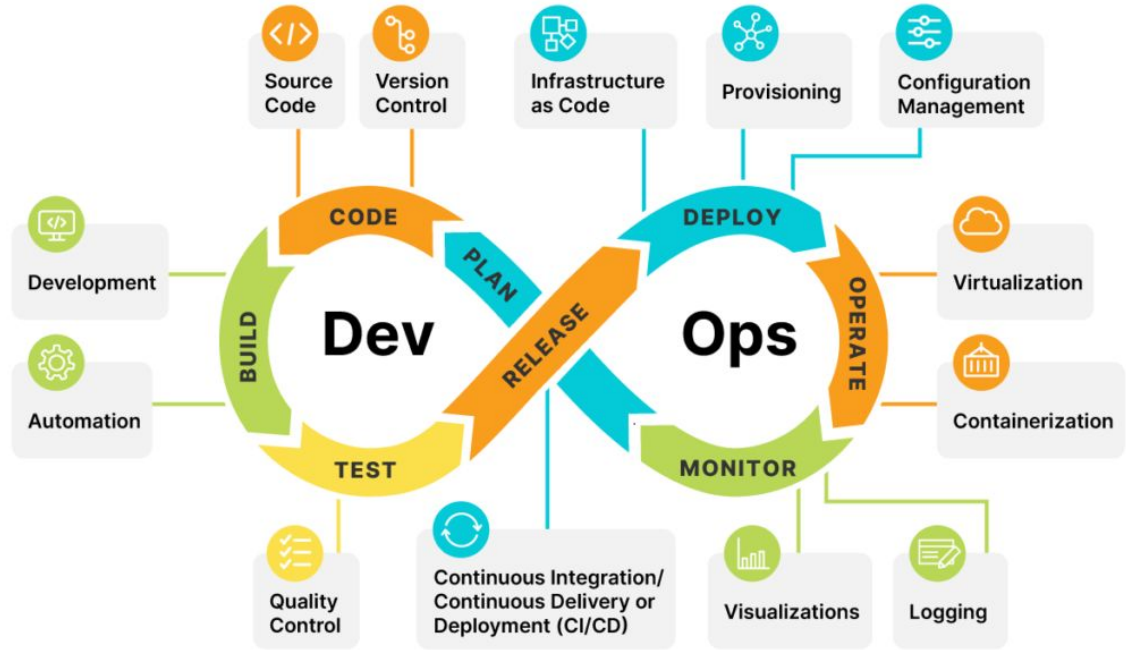
14

DevOps



DevOps

DevOps is a set of practices, tools, and a cultural philosophy that automate and integrate the processes between software development and IT teams. It emphasizes team empowerment, cross-team communication and collaboration, and technology automation.



Why DevOps?



Speed

Teams that practice DevOps release deliverables more frequently, with higher quality and stability. In fact, the DORA 2019 State of DevOps report found that elite teams deploy 208 times more frequently and 106 times faster than low-performing teams. Continuous delivery allows teams to build, test, and deliver software with automated tools.



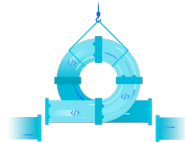
Improved collaboration

The foundation of DevOps is a culture of collaboration between developers and operations teams, who share responsibilities and combine work. This makes teams more efficient and saves time related to work handoffs and creating code that is designed for the environment where it runs.



Rapid deployment

By increasing the frequency and velocity of releases, DevOps teams improve products rapidly. A competitive advantage can be gained by quickly releasing new features and repairing bugs.



Quality and reliability

Practices like continuous integration and continuous delivery ensure changes are functional and safe, which improves the quality of a software product. Monitoring helps teams keep informed of performance in real-time.

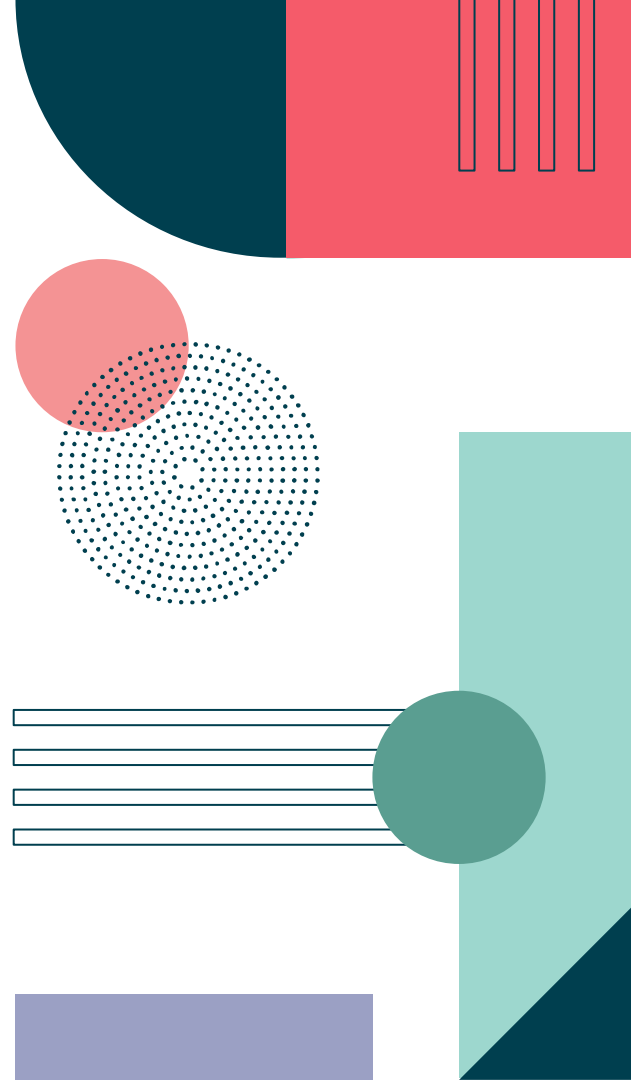


Security

By integrating security into a continuous integration, continuous delivery, and continuous deployment pipeline, DevSecOps is an active, integrated part of the development process. Security is built into the product by integrating active security audits and security testing into agile development and DevOps workflows.

15

Design Thinking

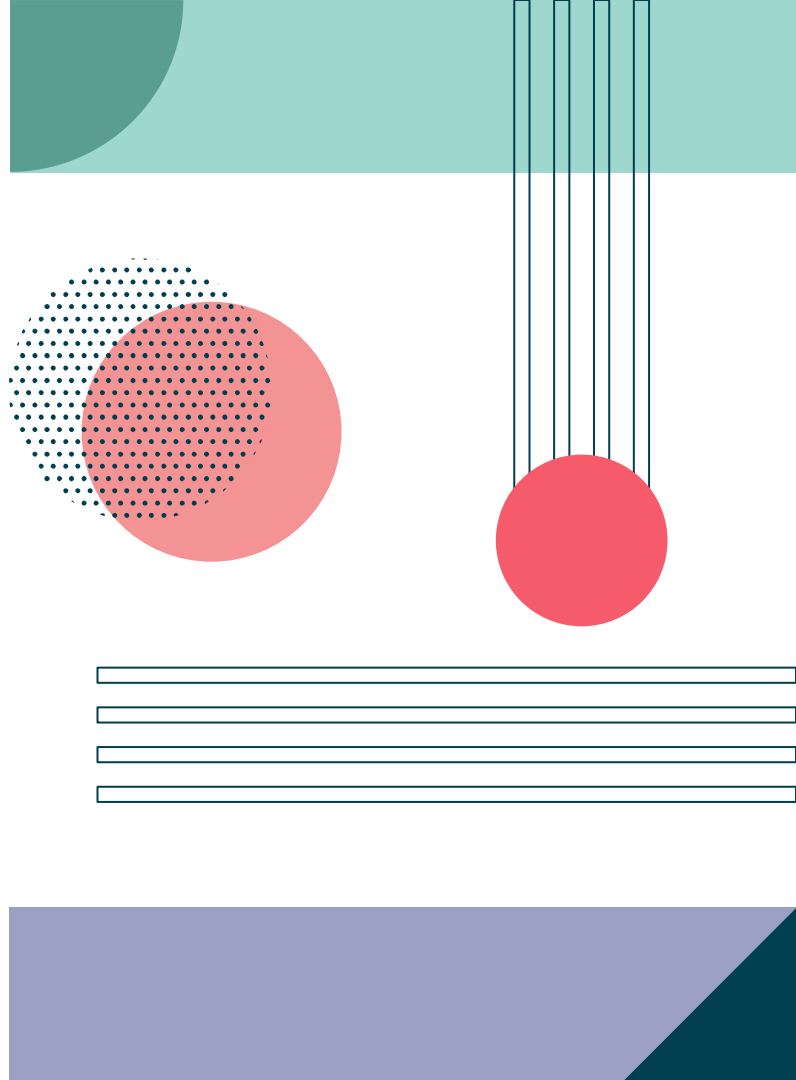


Design Thinking

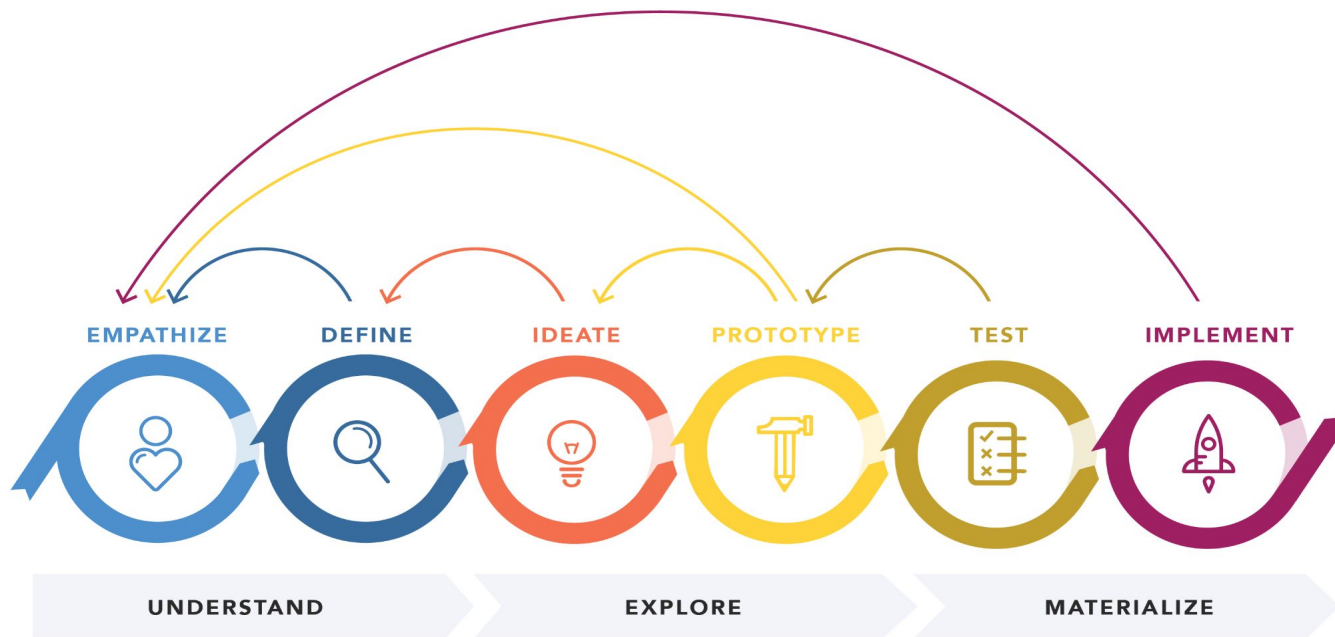
Design thinking is a non-linear, iterative process that teams use to understand users, challenge assumptions, redefine problems and create innovative solutions to prototype and test.

Design teams use design thinking to tackle ill-defined/unknown problems (aka wicked problems) because they can reframe these in human-centric ways and focus on what's most important for users.

Of all design processes, design thinking is almost certainly the best for “thinking outside the box”. With it, teams can do better UX research, prototyping and usability testing to uncover new ways to meet users’ needs.

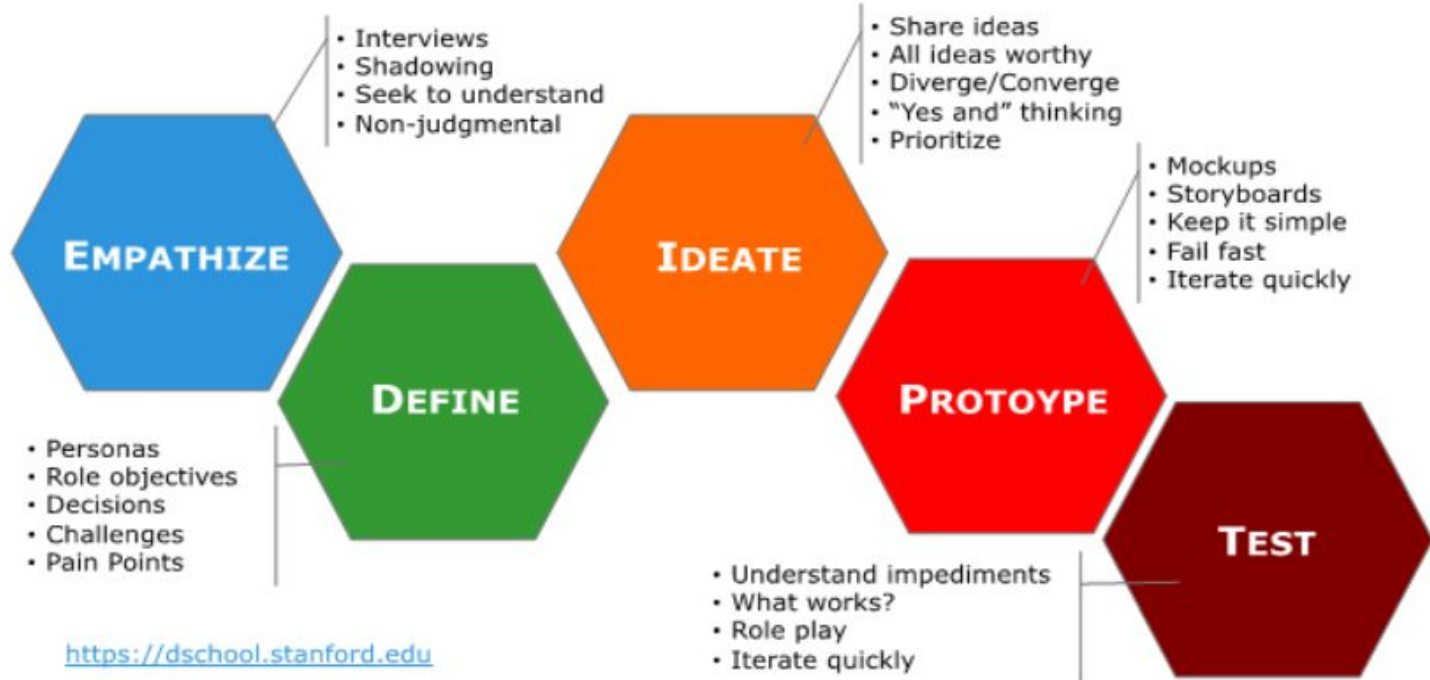


Stages of Design Thinking



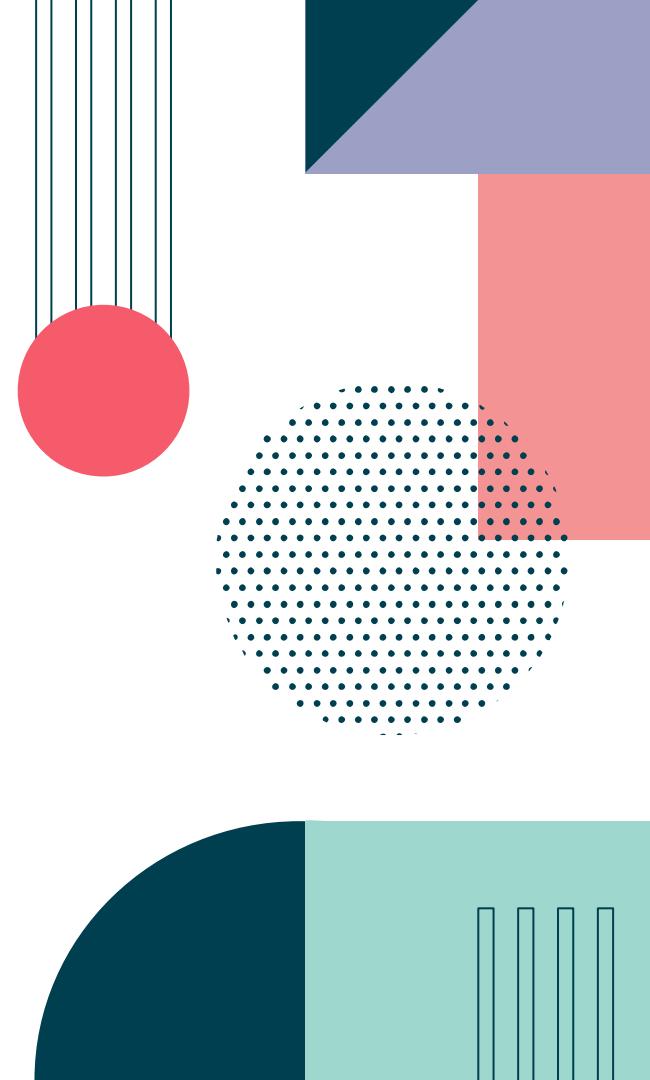
Detailed Stages

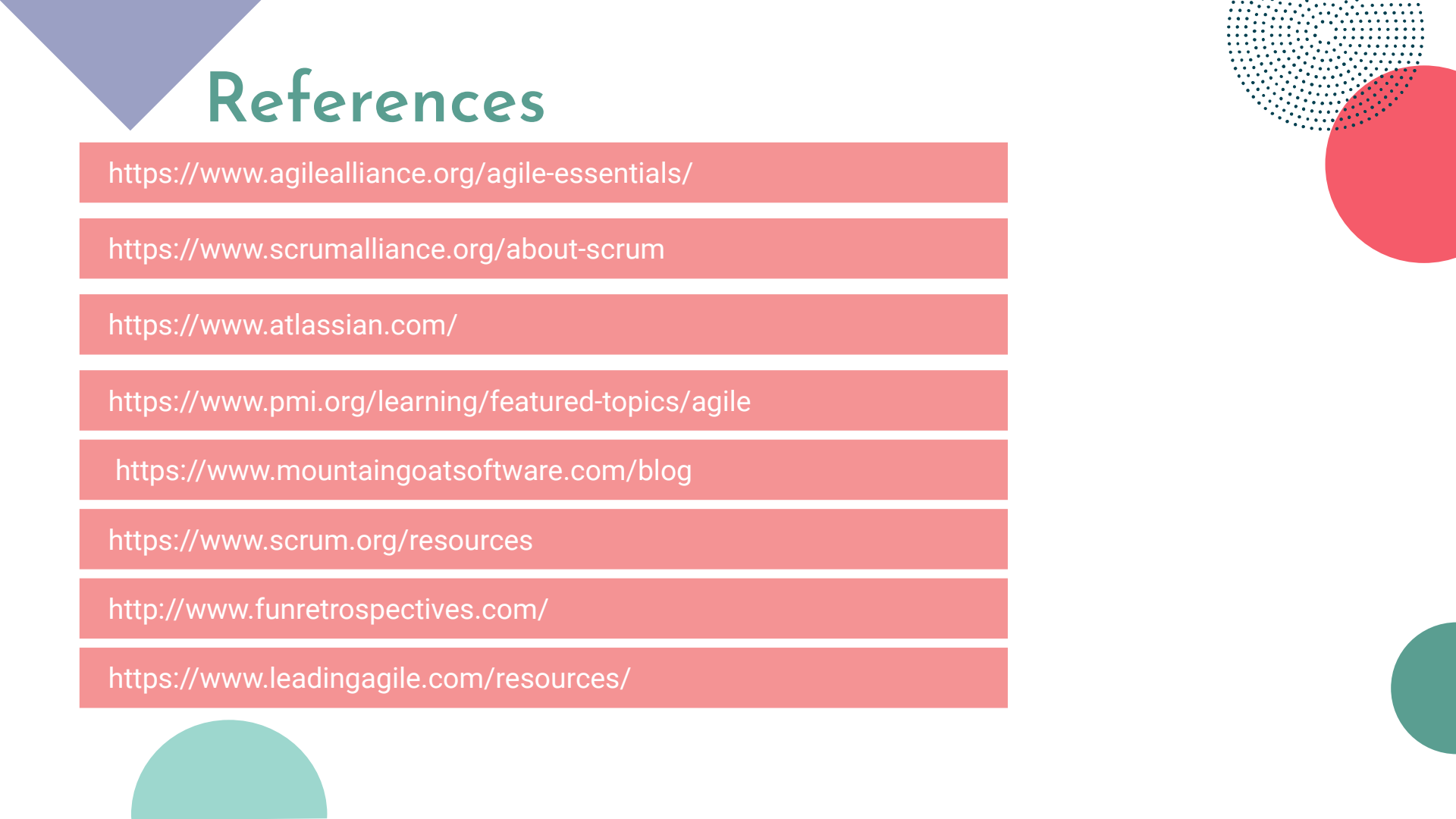
Stanford d.school Design Thinking Process



16

References





References

<https://www.agilealliance.org/agile-essentials/>

<https://www.scrumalliance.org/about-scrum>

<https://www.atlassian.com/>

<https://www.pmi.org/learning/featured-topics/agile>

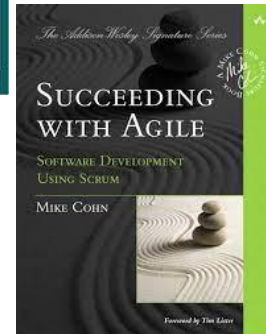
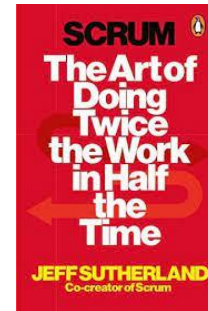
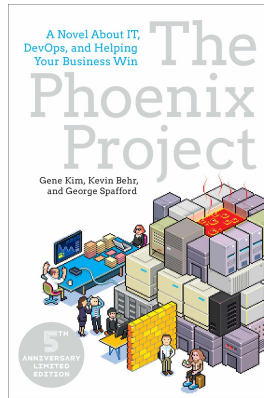
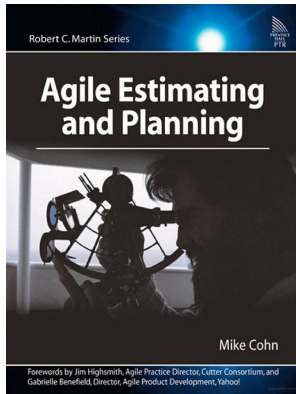
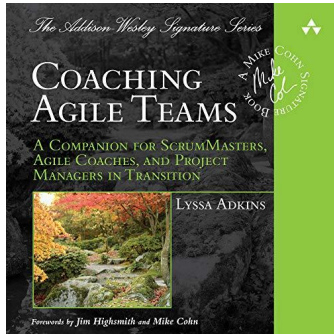
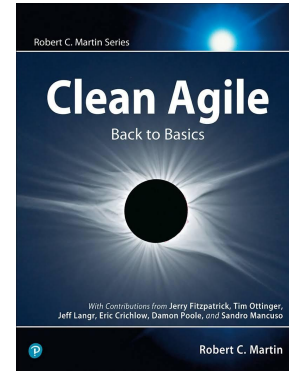
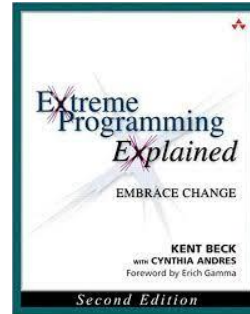
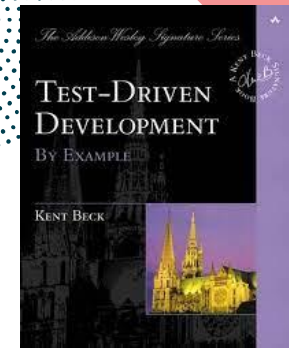
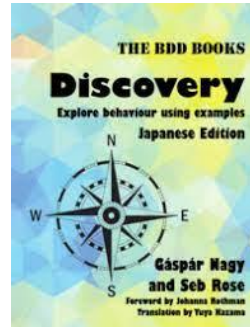
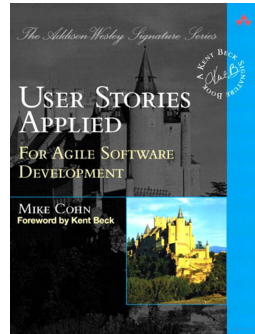
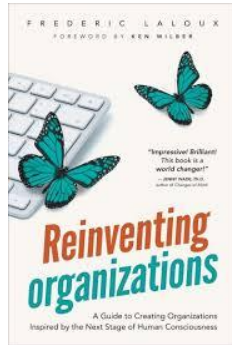
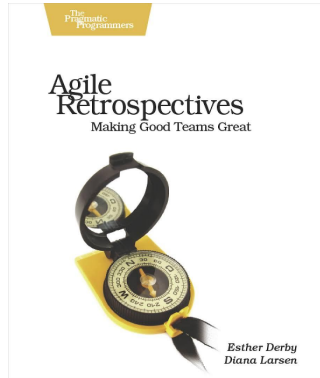
<https://www.mountangoatsoftware.com/blog>

<https://www.scrum.org/resources>

<http://www.funretrospectives.com/>

<https://www.leadingagile.com/resources/>

Books





Q & A

Thank You!

Do you have any questions?

elatfy.heba@gmail.com

+2 0122849910



<https://www.linkedin.com/in/heba-elatfy/>

